Agent Communication Protocols: MCP, A2A, and Google ADK

Building Multi-Agent Systems with MCP & Google ADK

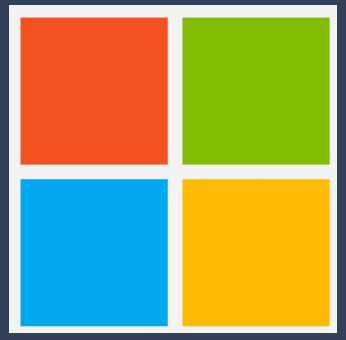
From naive chatbots to production-ready negotiation agents

{ik} | INTERVIEW KICKSTART

Introduction - Jeevendra Singh



- I have been working as a Software Engineer since 2015
- I've worked for companies like Ericsson, SAP, Q2, and now Microsoft
- I have been working at Microsoft for 4+ years
- In my career, I've worked as a full-stack developer, integration architect, and now focusing on Agentic AI development
- Built intelligent API platforms and copilots using Generative AI, RAG, and Agent Orchestration
- Trained over 2000+ learners on Agentic AI, Full-stack, DSA and System Design







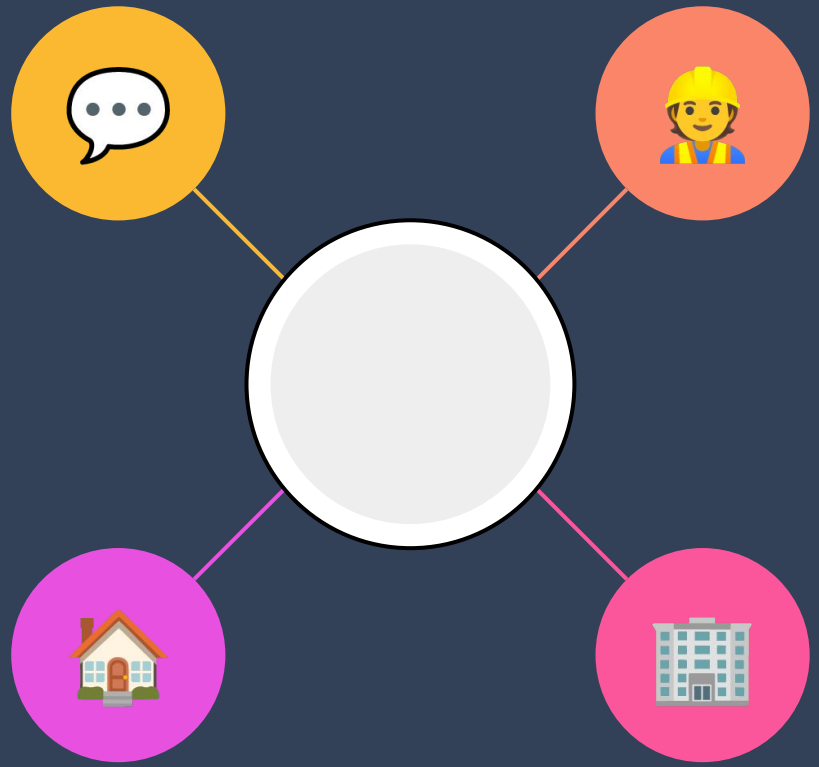


{ik} INTERVIEW KICKSTART

Welcome to today's class, before we begin, pop into the chat:

Hannah Chen

Data Scientist



Bay Area

Grammarly, ex-Uber

{ik} | INTERVIEW KICKSTART

Structure of Class

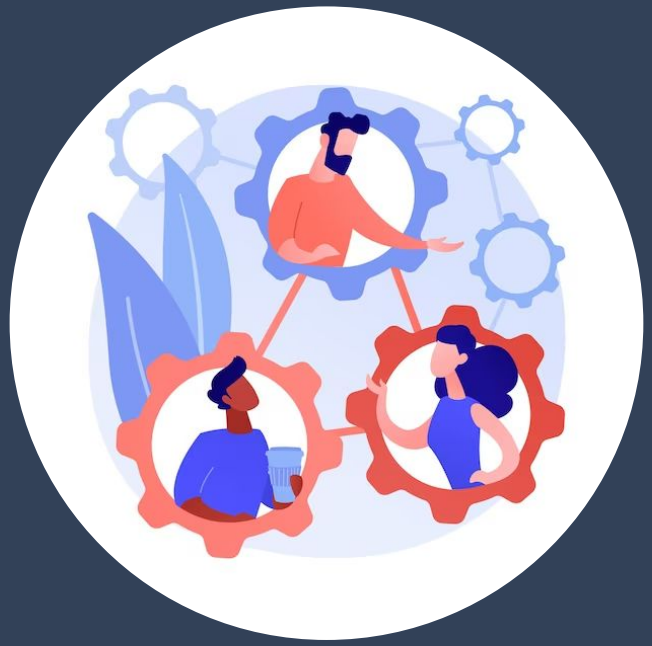
1. **Sunday Class - 4 hr (Sunday)**
2. **Assignments -** MCQ & Coding Assignment





{ik} | INTERVIEW KICKSTART

Optimize Your Experience



- ✅ Interact with your instructors via Sunday live class.
- ✅ Don’t be shy to speak up and get clarifications !



- ✅ Solve your assignments, MCQs and other assessments and get feedback (Thursday review session)
- ✅ Use your resources! It’s your experience – what you put in is what you’ll get out.

{ik} | INTERVIEW KICKSTART

Don’t worry!

We are here to help at your every step !

But, how???



{ik} | INTERVIEW KICKSTART

Are you using all these IK support features?

- **Uplevel** - contains all the learning resources - videos, MCQs, assignments
- Optional **Post class videos** for additional reference
- **Technical coaching sessions** on Wednesday to clarify all your queries
- **Discord** community for peer learning
- **TAs** to help answer the queries in the live class
- **Support tickets** - Raise them any time you want any clarifications/ tech concerns , etc on UpLevel

{ik} | INTERVIEW KICKSTART

Success Hacks

- Be prepared for the class by watching the pre class videos
- Attend every class
- Participate in class
- Attempt all the assignments,
- Actively advocate for yourself
- Keep your goal in mind
- Consistency is the key

Be patient with yourself !

{ik} | INTERVIEW KICKSTART

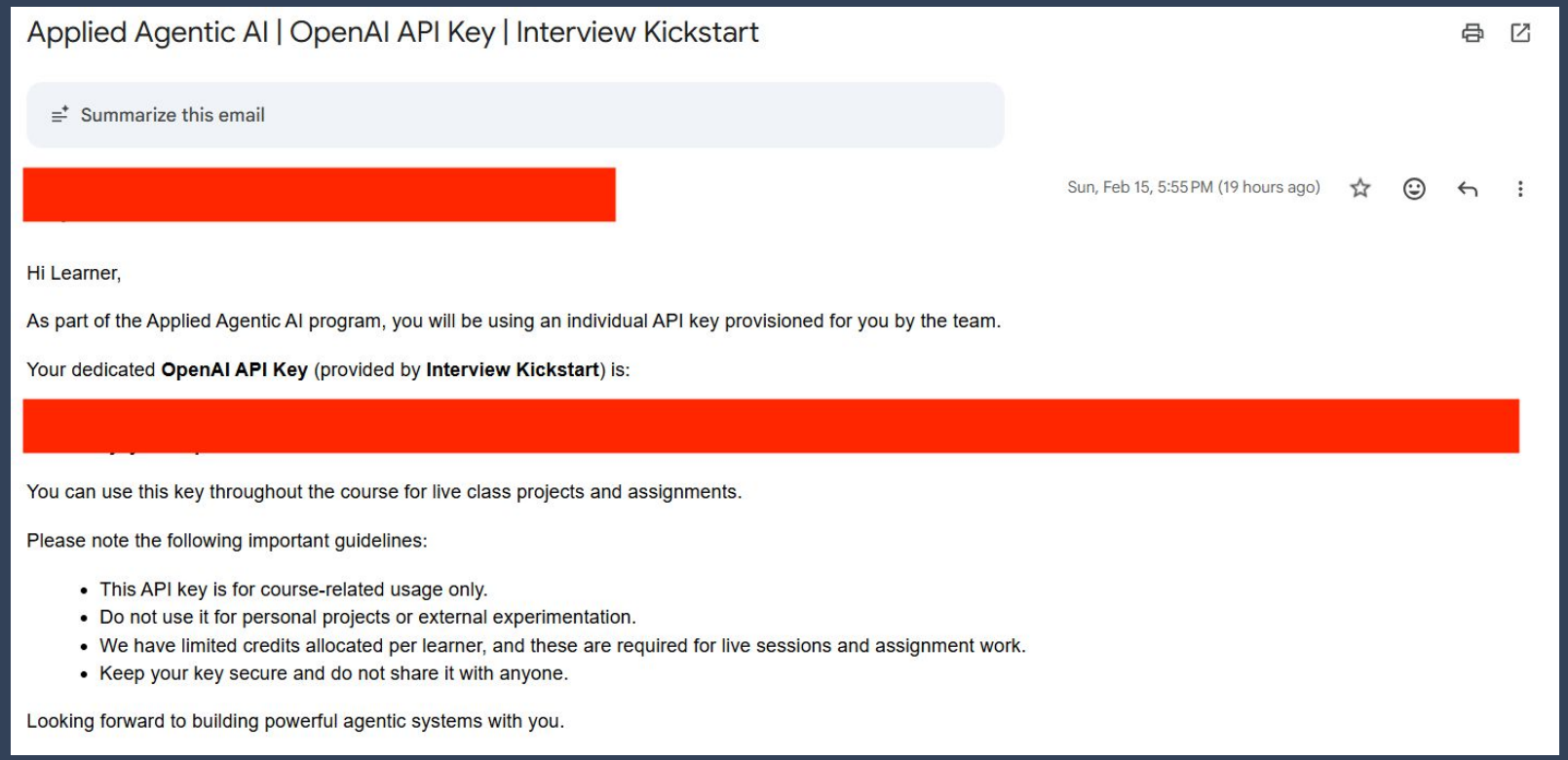
Check Your Email for API Key

You should have received an email like the one shown below containing your OpenAI API key.

If you do not see this email:

- Please check your inbox
- Also check your spam or junk folder

If you still cannot find it, please reach out to the Operations Team operations@interviewkickstart.com



Applied Agentic AI | OpenAI API Key | Interview Kickstart

Summarize this email

[redacted] Sun, Feb 15, 5:55 PM (19 hours ago)

Hi Learner,

As part of the Applied Agentic AI program, you will be using an individual API key provisioned for you by the team.

Your dedicated **OpenAI API Key** (provided by **Interview Kickstart**) is:

[redacted]

You can use this key throughout the course for live class projects and assignments.

Please note the following important guidelines:

- This API key is for course-related usage only.
- Do not use it for personal projects or external experimentation.
- We have limited credits allocated per learner, and these are required for live sessions and assignment work.
- Keep your key secure and do not share it with anyone.

Looking forward to building powerful agentic systems with you.

{ik} INTERVIEW KICKSTART

Clone & Practice: Your Learning Repository

real-estate-negotiation-simulator

https://github.com/jeev1992/real-estate-negotiation-simulator/

What's Included:

2 Complete Modules
Naive Baseline, MCP Servers, A2A & Google ADK

Working Code Examples
Ready-to-run demos, practice exercises, and solutions for each module

Getting Started

```
git clone
https://github.com/jeev1992/real-estate-negotiation-
simulator/
```

1. Install dependencies: `pip install -r requirements.txt`
2. Configure OpenAI API key and GitHub PAT
3. Run demos from each module directory

How to Follow Along

Setup & Running the Demos

```
# Clone and set up
git clone <repo> && cd real-estate-negotiation-simulator
python -m venv .venv && .venv\Scripts\activate    # Windows
pip install -r requirements.txt
cp .env.example .env   # add your OPENAI_API_KEY

# Module 1 -- MCP demos
python m1_mcp/demos/01_initialize_handshake.py
python m1_mcp/demos/02_tool_loop_trace.py
python m1_mcp/demos/03_list_all_primitives.py
python m1_mcp/pricing_server.py                    # start custom MCP server
python m1_mcp/github_agent_client.py "Find MCP servers"
python m1_mcp/sse_agent_client.py                  # agent over HTTP/SSE

# Module 2 -- ADK + A2A interactive
adk web m2_adk_multiagents/adk_demos/d01_basic_agent/
adk web m2_adk_multiagents/negotiation_agents/
adk web --a2a m2_adk_multiagents/negotiation_agents/
python m2_adk_multiagents/a2a_13_orchestrated_negotiation.py
```

Prerequisites: **Python 3.11+, OPENAI_API_KEY, Node.js (for GitHub MCP server only)**

Each module has a README.md with per-demo instructions.

{ik} | INTERVIEW KICKSTART

What exactly are we building?

{ik} | INTERVIEW KICKSTART

Problem statement

In Week 6, We are building a **real estate negotiation simulator** where two AI agents - a Buyer and a Seller - autonomously negotiate the purchase of 742 Evergreen Terrace, Austin TX 78701 (4 BR / 3 BA / 2,400 sqft / Listed at $485,000). The negotiation runs for a maximum of 5 rounds and must produce either a deal or a clean deadlock.

The Agents and Their Constraints :

Party	Goal	Starting Position	Walk-Away
Buyer Agent (GPT-4o)	Buy at lowest price	Offer ~$425,000	Over $460,000
Seller Agent (GPT-4o)	Sell at highest price	Counter $477,000	Below $445,000

The problem we're solving (in the real world)

Real estate negotiations happen every day, and they are complex because:

- A buyer at 742 Evergreen Terrace, Austin TX wants to pay as little as possible (budget: $460K)
- The seller won't go below their mortgage floor ($445K) but listed at $485K
- Both sides need live market data to justify every move
- The negotiation must terminate fairly within 5 rounds with either a deal or a clear deadlock

Today's Agenda

1



MCP - External Data for Agents

2



Google ADK + A2A Protocol

{ik} INTERVIEW KICKSTART

MCP: External Data for Agents

MCP Model Context Protocol is a standard that lets agents call external tools without knowing where the data comes from.

```
Agent: "What tools do you have?"          → list_tools
Server: "get_market_price, ..."           → tool schemas (JSON Schema)
Agent: "Call get_market_price(...)"       → call_tool
Server: {estimated_value: 462000}         → structured result
```

By the end of this module:

- You'll understand the MCP protocol (handshake, tools, resources, prompts)
- You'll have seen a live agent use GitHub MCP tools
- You'll have built and inspected custom MCP servers
- You'll understand stdio vs SSE vs Streamable HTTP transports

{ik} | INTERVIEW KICKSTART

API vs CLI vs MCP: Three Paths to External Systems

> *"Agents are only as useful as the systems they can reach."* - Anthropic

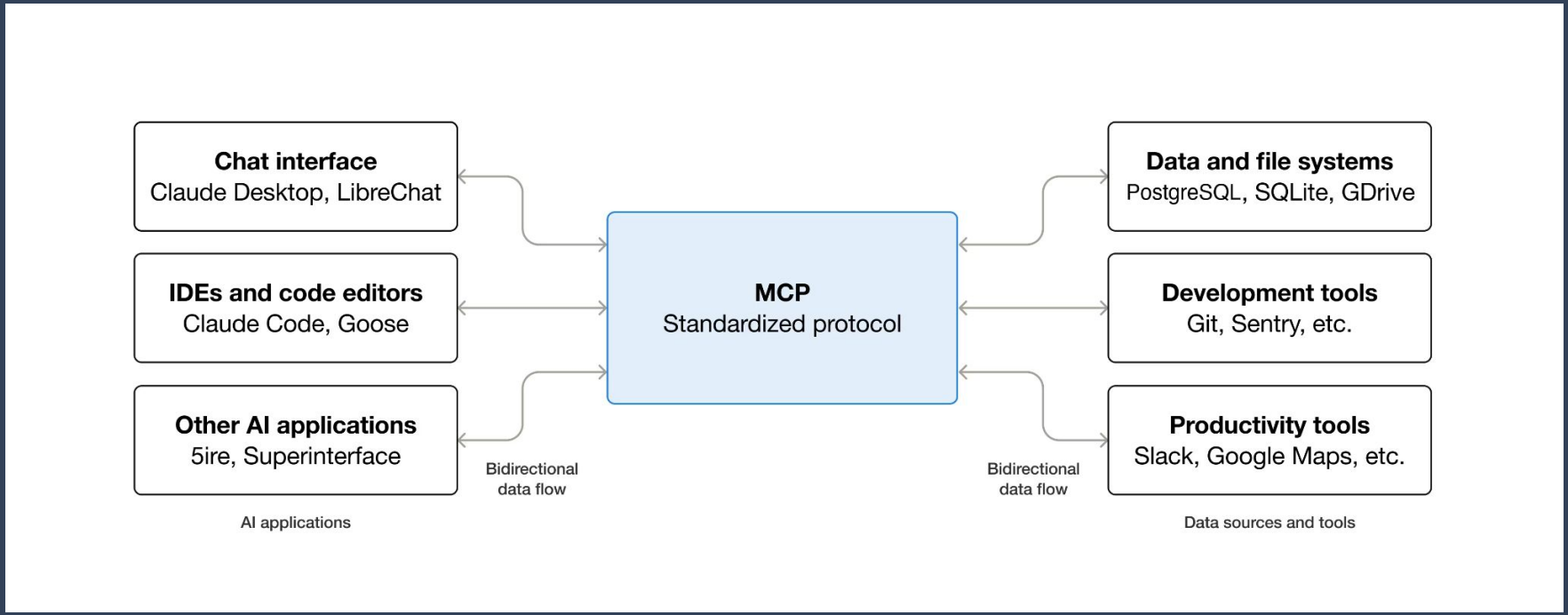
Path	How it works	Limitation
Direct API	Agent calls HTTP endpoints directly	M×N integration problem - each pair is bespoke
CLI	Agent runs shell commands	No reach to web/mobile/cloud - needs filesystem
MCP	Standard protocol - discovery + invocation + auth	Upfront investment, but portable across all clients

WHY MCP WINS FOR PRODUCTION

- One remote server reaches any compatible client (Claude, ChatGPT, Cursor, VS Code, ADK)
- Auth, discovery, and rich semantics are standardized
- 300M+ SDK downloads/month
- Build once, use everywhere

{ik} | INTERVIEW KICKSTART

What is the Model Context Protocol (MCP) ?



- MCP (Model Context Protocol) is an open-source standard for connecting AI applications to external systems.
- Using MCP, AI applications like Claude or ChatGPT can connect to data sources (e.g. local files, databases), tools (e.g. search engines, calculators) and workflows (e.g. specialized prompts) - enabling them to access key information and perform tasks.
- Think of MCP like a USB-C port for AI applications. Just as USB-C provides a standardized way to connect electronic devices, MCP provides a standardized way to connect AI applications to external systems.

{ik} | INTERVIEW KICKSTART

MCP: Core Operations and Benefits

MCP (Model Context Protocol) is a thin RPC layer over **JSON-RPC 2.0** that standardizes three operations between an agent host and an external tool server.

1. **initialize**
   Handshake, version and capability negotiation
2. **tools/list**
   The server advertises typed tool schemas
3. **tools/call**
   The host invokes a tool by name with validated arguments

Everything else - resources, prompts, streaming - is layered on this base.

Why it matters

N + M not N × M

One pricing server wired once, used by every agent in your stack regardless of language.

Out-of-process tools

A tool crash does not crash the agent. The server is isolated compute.

Same shape you know

If you know OpenAI function calling or Anthropic tool use, MCP is the same shape - but the registry is reusable and lives out of process.

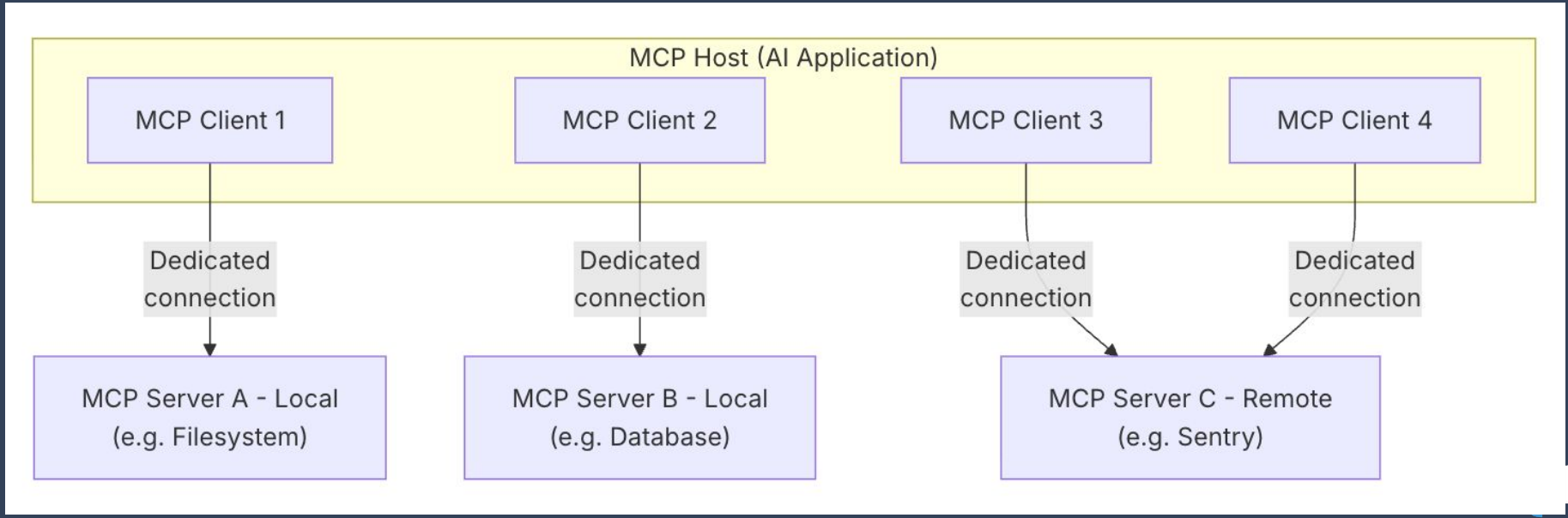
{ik} INTERVIEW KICKSTART

Participants

The key participants in the MCP architecture are:

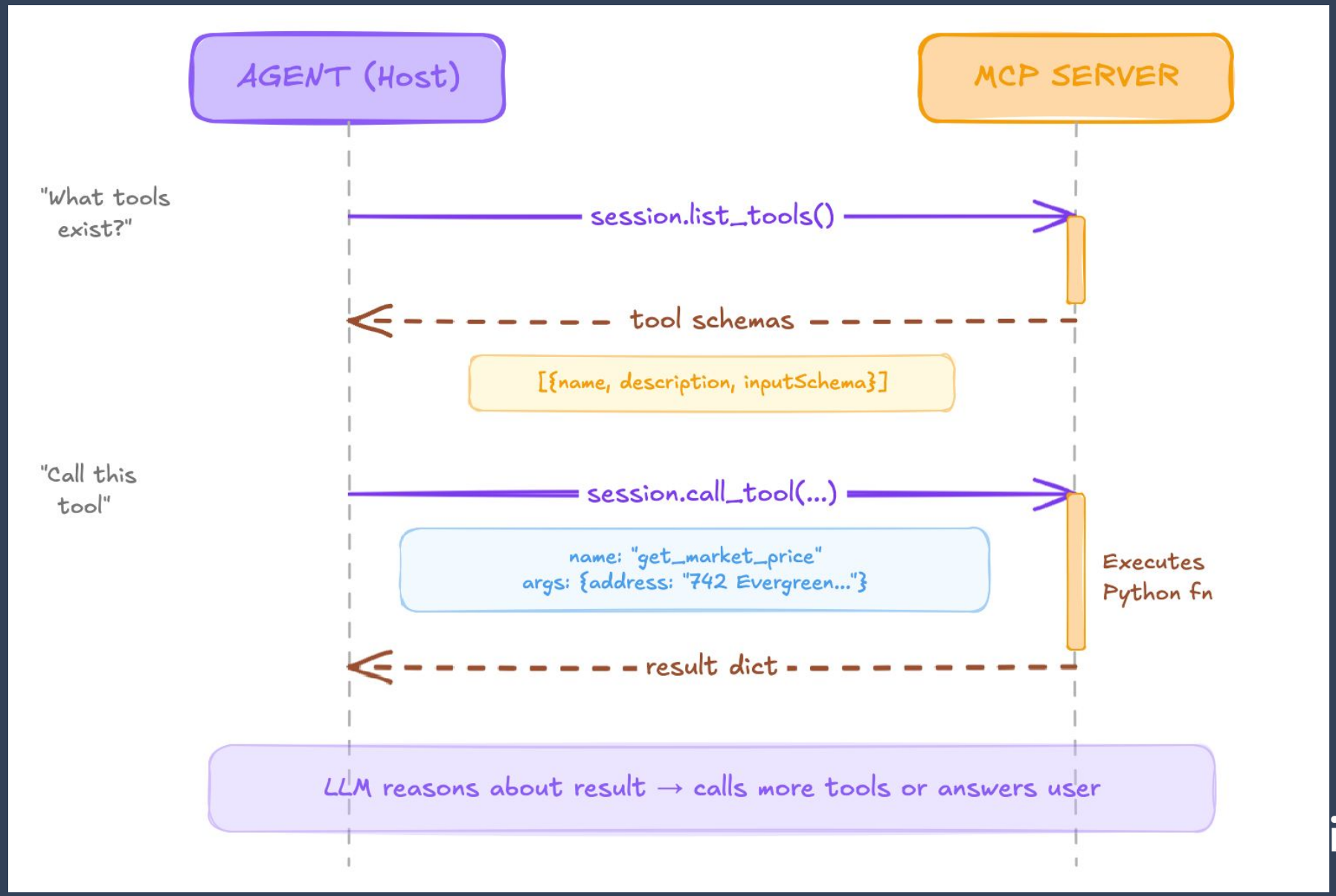
- **MCP Host:** The AI application that coordinates and manages one or multiple MCP clients
- **MCP Client:** A component that maintains a connection to an MCP server and obtains context from an MCP server for the MCP host to use
- **MCP Server:** A program that provides context to MCP clients

For example: Visual Studio Code acts as an MCP host. When Visual Studio Code establishes a connection to an MCP server, such as the Sentry MCP server, the Visual Studio Code runtime instantiates an MCP client object that maintains the connection to the Sentry MCP server. When Visual Studio Code subsequently connects to another MCP server, such as the local filesystem server, the Visual Studio Code runtime instantiates an additional MCP client object to maintain this connection.



INTERVIEW KICKSTART

MCP: Agent ↔ Server Sequence



ik} | INTERVIEW KICKSTART

MCP Primitives & Content Types: Four Primitives, Three Content Types

Primitive	Direction	Decorator	When to pick it
Tool	Model invokes with arguments	@mcp.tool()	Side-effecting action that needs parameters
Resource	Host fetches by URI	@mcp.resource("scheme://path")	Read-only document, static or computed
Prompt	Host selects template	@mcp.prompt("name")	Parameterized reusable instructions
Sampling	Server asks host to call LLM	(host-side capability)	Server needs LLM reasoning without holding API keys

Tools are 90% of MCP usage. Resources and prompts are bonus capabilities.

CONTENT TYPES - WHAT A TOOL CAN RETURN

Type	JSON shape	When to use
TextContent	{"type": "text", "text": "..."}	Most common - tool results
ImageContent	{"type": "image", "data": "base64..."}	Charts, screenshots
EmbeddedResource	{"type": "resource", "resource": {...}}	References to stored data

{ik} | INTERVIEW KICKSTART

Three Transports: stdio, SSE, Streamable HTTP

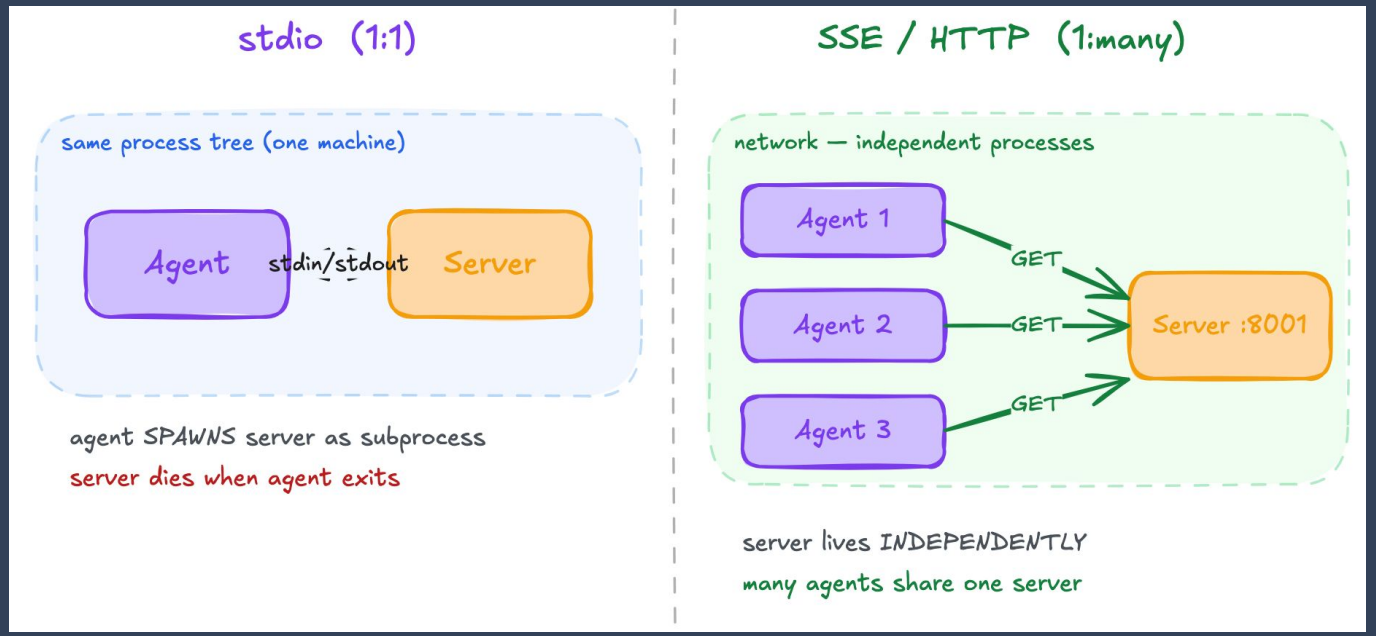
Same protocol, different carrier. Switch with a runtime flag - tool functions change nothing.

Transport	Shape	When to use
stdio	Subprocess pipes - lowest latency	Local dev, integration tests, single-tenant desktop apps. Spawn the server, pipe stdin/stdout, kill on exit.
SSE	Server-sent events over HTTP	Legacy remote deployments. Two endpoints (/sse + /messages). Use only if you inherited it.
Streamable HTTP	HTTP with chunked SSE response body	Current spec preferred option for remote/multi-tenant. Single endpoint, streaming on the same connection. Use this for new deployments.

```python
# stdio - subprocess, single tenant
mcp.run()                                   # default


# Streamable HTTP - multi tenant, new standard
mcp.settings.host = "0.0.0.0"
mcp.settings.port = 8001
mcp.run(transport="streamable-http")        # preferred for
remote


# SSE - legacy, avoid for new servers
mcp.run(transport="sse")
```



{ik} INTERVIEW KICKSTART

Demo 01: The MCP Handshake

What Happens on the Wire

```
python m1_mcp/demos/01_initialize_handshake.py   # no API key needed
```

Raw JSON-RPC frames - no SDK, no abstraction:

```
>>> client → server: initialize
    {"jsonrpc": "2.0", "method": "initialize",
     "params": {"protocolVersion": "2024-11-05", "capabilities": {},
                "clientInfo": {"name": "demo-client", "version": "0.1"}}}

<<< server → client: initialize result
    {"result": {"protocolVersion": "2024-11-05",
                "capabilities": {"tools": {}, "prompts": {}, "resources": {}},
                "serverInfo": {"name": "real-estate-pricing", "version": "1.27.0"}}}

>>> notifications/initialized
>>> tools/list
<<< tools/list result → 2 tools with full JSON schemas
```

5 frames total. After this handshake, the client knows what tools exist and can call them. JSON Schema is generated from Python type hints - no manual schema writing.

{ik} | INTERVIEW KICKSTART

Demo 02: The Tool Loop

Model ↔ Host ↔ Server

```
python m1_mcp/demos/02_tool_loop_trace.py   # requires OPENAI_API_KEY
```

```
[t= 0.00s] HOST    connecting to MCP server (stdio subprocess)
[t= 3.18s] HOST    tools/list → 2 tools discovered
[t= 3.67s] MODEL   receiving prompt + tool catalog
[t= 6.77s] MODEL   emitted tool_use(get_market_price)
[t= 6.77s] HOST    translating tool_use → tools/call
[t= 6.77s] SERVER  returned CallToolResult
[t= 6.77s] HOST    injecting tool_result into model context
[t=10.42s] MODEL   emitted final assistant text
```

Three actors, clear roles:

Actor	What it does
Model (GPT-4o)	Decides WHICH tools to call based on query + schemas
Host (your Python code)	Translates between OpenAI format and MCP format
Server (pricing_server.py)	Executes the tool and returns structured data

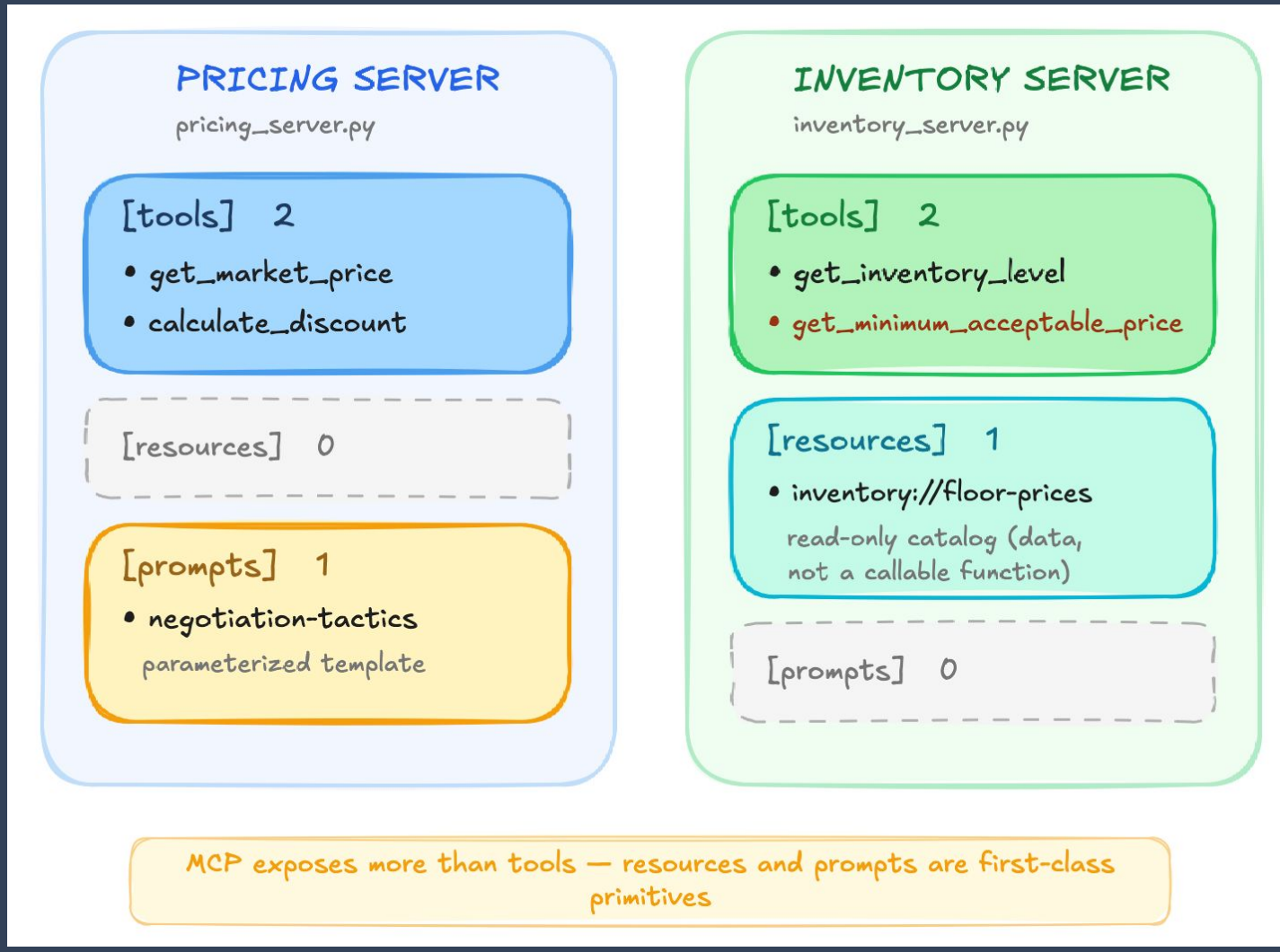
Demo 03: List All Primitives

Tools, Resources, and Prompts

```
python m1_mcp/demos/03_list_all_primitives.py   # no API key needed
```

What to point out live:

- MCP exposes more than tools; resources and prompts are first-class primitives too
- inventory://floor-prices is readable data, not a callable function
- negotiation-tactics is a prompt template the host can expand into the conversation



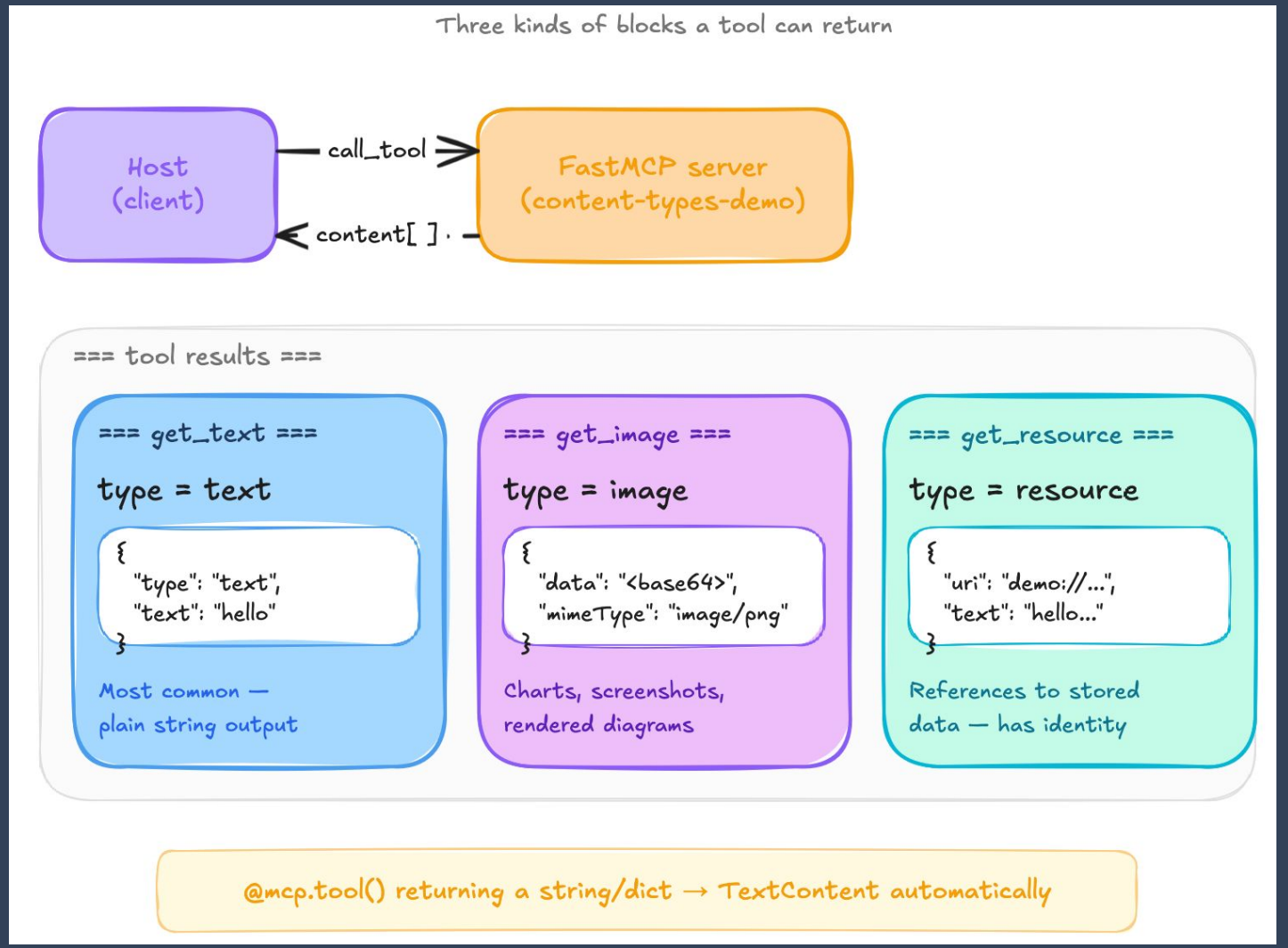
Demo 04: Content Types

Text, Image, and Embedded Resource

```
python m1_mcp/demos/04_content_types.py   # no API key needed
```

Why it matters:

- Most workshop tools return TextContent, but MCP can also return images and embedded resources
- The protocol standardizes result blocks, so the host can render them consistently
- Demo 04 makes the JSON shape visible before students ever need to debug it in Module 2.



GitHub MCP Agent (Live Demo) : An LLM Agent Using GitHub via MCP

```
python m1_mcp/github_agent_client.py "Find popular MCP server implementations"
```

WHAT HAPPENS

- Agent spawns GitHub's MCP server via npx @modelcontextprotocol/server-github
- Discovers 20+ tools (search repos, read files, list issues, etc.)
- GPT-4o reads your query, picks tools, calls them via MCP
- Feeds results back, calls more tools or produces final answer

THE AGENTIC LOOP

```python
for iteration in range(max_iterations):
    response = await openai.create(messages=messages, tools=openai_tools)
    if response.tool_calls:
        for call in response.tool_calls:
            result = await session.call_tool(call.name, call.args)  # MCP
            messages.append({"role": "tool", "content": result})
    else:
        return response.content  # final answer
```

This is the pattern ADK automates. In Module 2, MCPToolset + LlmAgent replaces this entire loop with one declaration: tools=[MCPToolset(...)].

{ik} | INTERVIEW KICKSTART

Custom MCP Servers

Building Your Own MCP Server

```python
# pricing_server.py - 5 lines to expose a tool
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("real-estate-pricing")

@mcp.tool()
def get_market_price(address: str, property_type: str = "single_family") -> dict:
    """Get market price analysis for a property."""
    return {"estimated_value": 462_000, "comparable_sales": [...], ...}
```

WHAT @MCP.TOOL() DOES

- Reads the function signature → JSON Schema (inputSchema)
- Reads the docstring → tool description
- Registers as a callable MCP tool

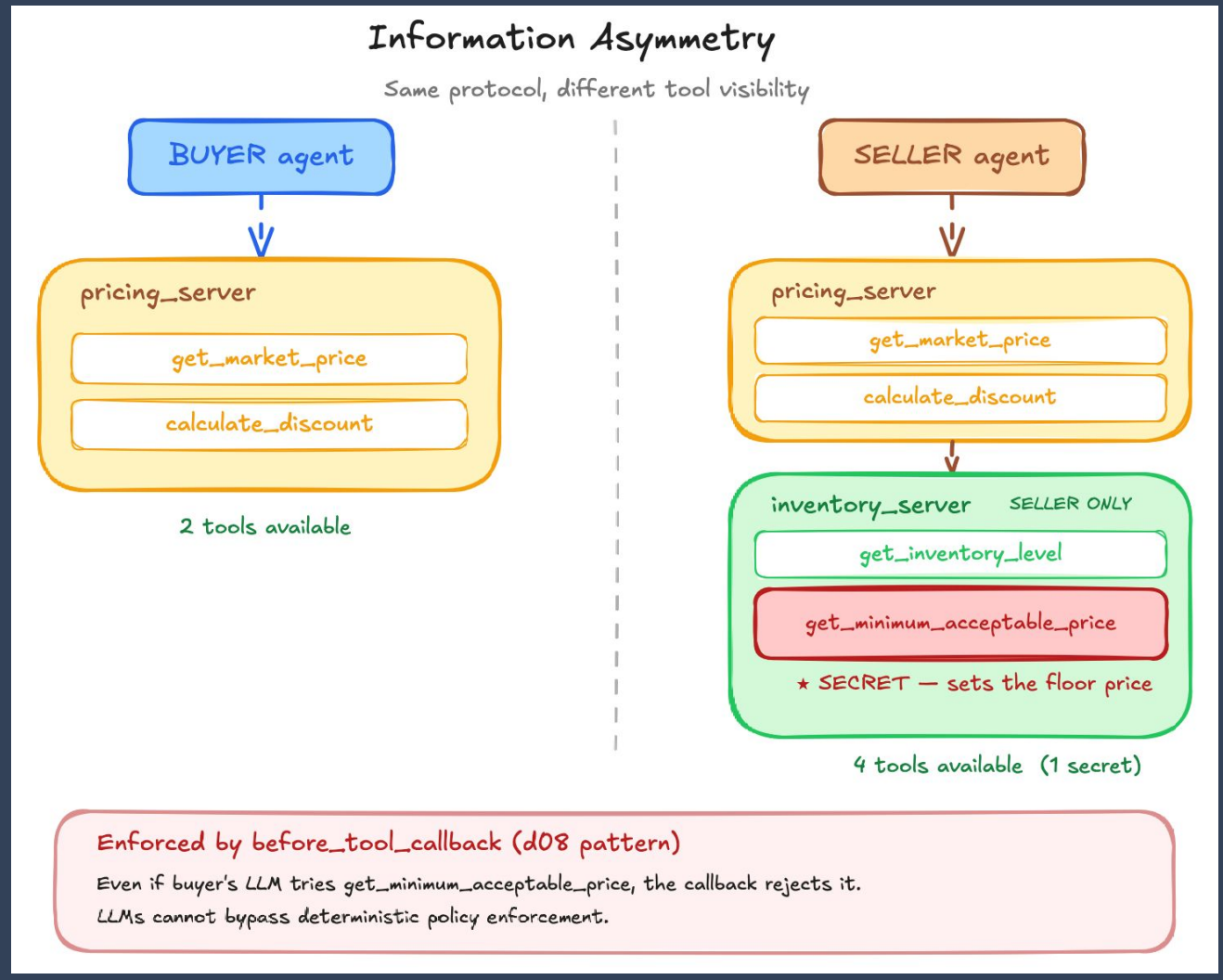
> *"Group tools around intent, not endpoints."* A single get_market_price beats get_comps + get_sqft + get_days_on_market - Anthropic

Server	Tools	Who uses it (Module 2)
pricing_server.py	get_market_price, calculate_discount	Both buyer and seller
inventory_server.py	get_inventory_level, get_minimum_acceptable_price	Seller only

{ik} | INTERVIEW KICKSTART

Information Asymmetry via MCP: Different Agents, Different Tools

The floor price ($445K) lives in the server, not the code.



- M1: get_minimum_acceptable_price() - returned by MCP at runtime, only accessible to seller (enforced in Module 2)

This is how real production systems work. Different agents get different tool access. The protocol is the same - the authorization is different.

{ik} INTERVIEW KICKSTART

SSE MCP Agent (Live Demo): Same Loop, HTTP Transport

```
# Terminal 1: start MCP servers in SSE mode
python m1_mcp/pricing_server.py --sse --port 8001
python m1_mcp/inventory_server.py --sse --port 8002

# Terminal 2: run the agent
python m1_mcp/sse_agent_client.py "What is 742 Evergreen Terrace worth?"
# With both servers:
python m1_mcp/sse_agent_client.py --both "What's the seller's minimum price?"
```

	github_agent_client	sse_agent_client
Transport	stdio (subprocess)	SSE (HTTP)
Server	GitHub's MCP server (TypeScript)	pricing_server.py + inventory_server.py
Multi-server	No	Yes (--both merges 4 tools from 2 servers)
Tool loop	Identical	Identical

The agent loop is identical. Only the connection setup changes. This proves transport is pluggable - the agent doesn't care how it connects to the server.
--both reveals information asymmetry: with both servers, the agent can see the seller's $445K floor. In Module 2, the buyer's allowlist blocks this tool.

SSE vs stdio vs Streamable HTTP: Three Transports Side by Side

STDIO (DEMOS 01–04)

```
server_params = StdioServerParameters(command="python", args=["pricing_server.py"])
async with stdio_client(server_params) as (read, write):
    # server is a subprocess - dies when you disconnect
```

SSE (sse_agent_client.py)

```
# Terminal 1: start server
python m1_mcp/pricing_server.py --sse --port 8001
# Terminal 2: connect agent
python m1_mcp/sse_agent_client.py "What is 742 Evergreen Terrace worth?"
```

STREAMABLE HTTP (DEMO 05)

```
# Terminal 1: server
python m1_mcp/demos/05_streamable_http_transport.py --serve --port 8765
# Terminal 2: client
python m1_mcp/demos/05_streamable_http_transport.py --client --port 8765
```

The agent code is identical - only the connection setup changes. list_tools() and call_tool() work the same way regardless of transport.

Demo 05: Streamable HTTP Results

HTTP Transport in Action

```
# Terminal 1:
python m1_mcp/demos/05_streamable_http_transport.py --serve --port 8765
# Terminal 2:
python m1_mcp/demos/05_streamable_http_transport.py --client --port 8765
```

CLIENT OUTPUT

```
Connecting client to http://127.0.0.1:8765/mcp
discovered tools: ['echo']
response: echo: hello over HTTP
```

THE SERVER IS 5 LINES

```
mcp = FastMCP("http-transport-demo")

@mcp.tool()
def echo(text: str) -> str:
    """Echo back the supplied text."""
    return f"echo: {text}"

mcp.run(transport="streamable-http")
```

Same session.list_tools(), same session.call_tool(). Only the import changes. The server is an independent HTTP process - survives client disconnection, multiple clients can connect simultaneously.

{ik} | INTERVIEW KICKSTART

What MCP Does Not Solve for You

Idempotency

Retries after timeouts hit your server twice. A hung tool call on the host side does not mean the server did not execute. Design every tool to be safely retryable, or return an idempotency key the host can deduplicate on.

Cost accounting

MCP does not know that get_market_price costs $0.004 in provider fees. Wire telemetry at the host level and correlate tool name, duration, and downstream cost. Track this before your first production bill.

Determinism

Two calls with identical arguments can return different data if the backing system drifts. If the agent's reasoning must be reproducible, cache tool results or pin them to a timestamp-versioned snapshot.

Prompt injection defense

A tool result containing "ignore previous instructions" still lands in the model's context window. Sanitize or structure outputs before the host forwards them. Do not treat tool results as trusted system content.

Audit

The server log is not enough. Correlate the host-side JSON-RPC id with the server-side resource access log. Without both ends you cannot reconstruct what the model actually caused to happen.

Use MCP for the protocol shape. Put the rest of your production concerns in the host layer.

{ik} INTERVIEW KICKSTART

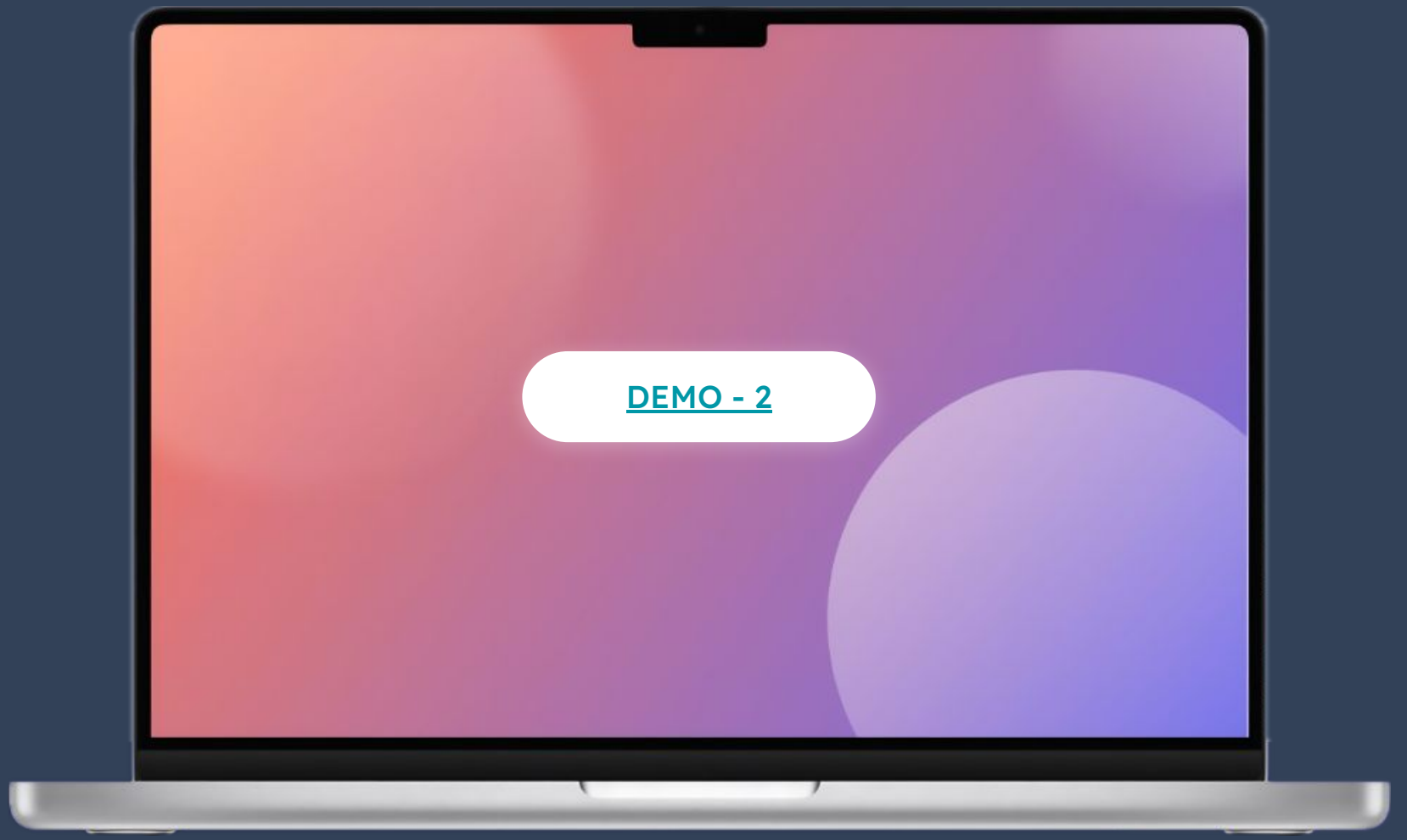
Module 1: What You Learned

Concept	What it means	Where we saw it
MCP Protocol	JSON-RPC handshake + tool discovery + invocation	Demo 01 (raw frames)
Three actors	Model (decides), Host (bridges), Server (executes)	Demo 02 (timestamped trace)
Four primitives	Tools, Resources, Prompts, Sampling	Demo 03 (list all)
Content types	TextContent, ImageContent, EmbeddedResource	Demo 04
Three transports	stdio (1:1), SSE (1:many), Streamable HTTP	Demos 01–04, 05
Tool design	Group around intent, not endpoints	get_market_price (one call)
Information asymmetry	Different agents see different tools	Buyer vs seller server access
Agentic loop	LLM + tool schemas → decide → call → reason	github_agent_client.py

WHAT CARRIES FORWARD TO Module 2

pricing_server.py and inventory_server.py are used directly by buyer/seller agents
The agentic loop pattern is replaced by MCPToolset + LlmAgent
Information asymmetry is enforced by before_tool_callback allowlists

Module 1 - Implementation Notes: MCP Deep Dive



{ik} | INTERVIEW KICKSTART

No Question Is Too Small Every Question Deserves an Answer!!

Ask about anything:

- Concepts from today's modules
- How this applies to your work
- Implementation details
- Clarifications on what we covered

Quick Request

- Please do not mind if a question gets skipped accidentally
- Repost politely in chat if needed
- We will try our best to cover as many questions as possible

Remember: If you're wondering about it, others probably are too.



Time for a Break



A 10-minute break is starting now. Please return when the timer ends to continue the presentation

{ik} | INTERVIEW KICKSTART

Module 1 → Module 2: What Changes

In Module 2, you write the tool loop manually (~60 lines):

```
tools = await session.list_tools()
openai_tools = mcp_tools_to_openai_functions(tools)
for iteration in range(max_iterations):
    response = await openai.create(messages=messages, tools=openai_tools)
    if response.tool_calls:
        for call in response.tool_calls:
            result = await session.call_tool(call.name, call.args)
            messages.append({"role": "tool", "content": result})
    else:
        return response.content
```

In Module 2, one line replaces all of that:

```
root_agent = LlmAgent(
    tools=[MCPToolset(connection_params=StdioConnectionParams(
        server_params=StdioServerParameters(command="python", args=["pricing_server.py"])
    ))]
)
```

Module 2 teaches you what's happening inside. Module 3 lets you stop writing it.

Today's Agenda

1



MCP - External Data for Agents

2



Google ADK + A2A Protocol

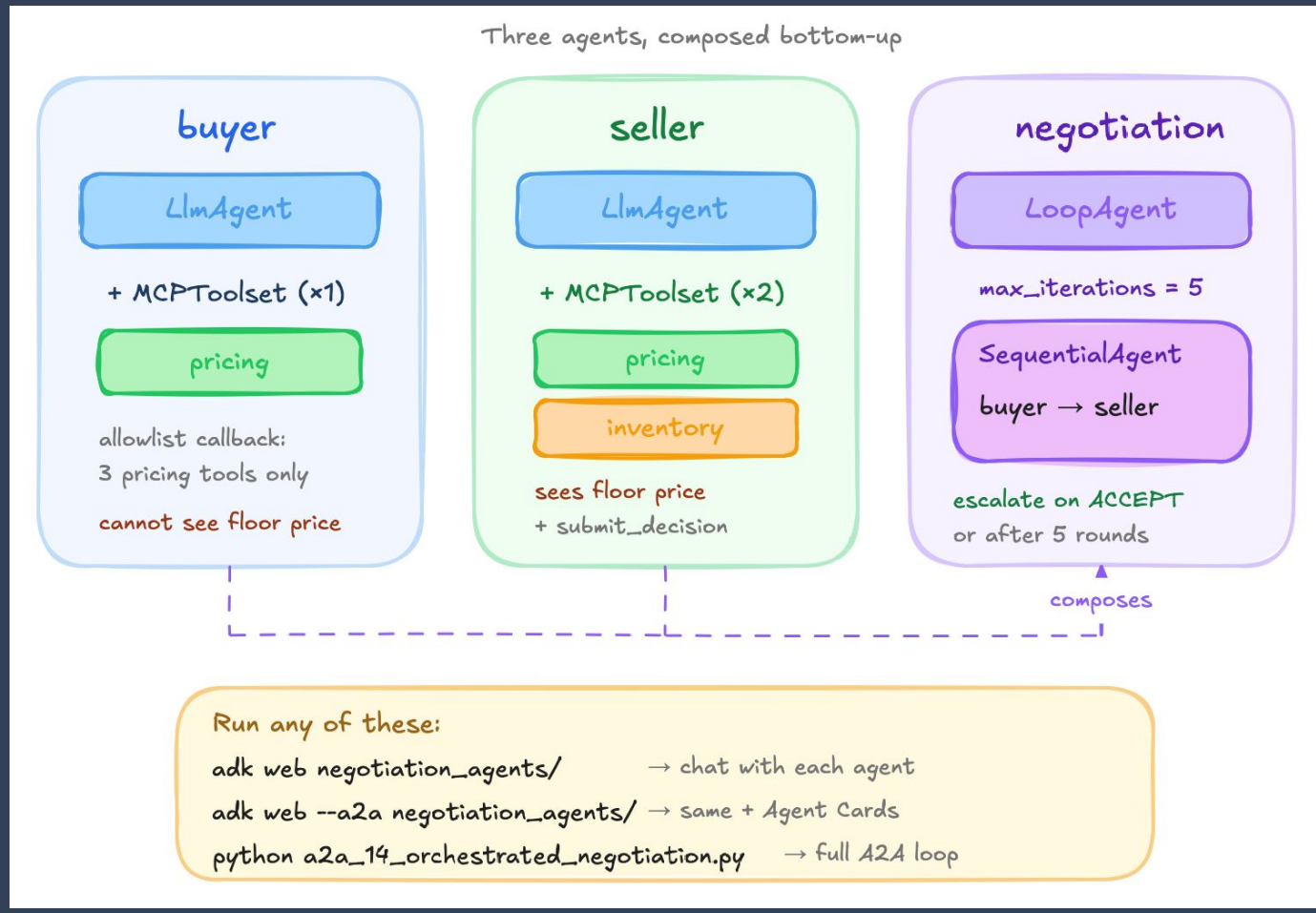
{ik} INTERVIEW KICKSTART

Google ADK + A2A Protocol

The goal: *Build a complete multi-agent negotiation system using Google ADK*

By the end of this module you'll have:

- A buyer agent with MCP pricing tools
- A seller agent with MCP pricing + inventory tools (information asymmetry)
- A negotiation orchestrator that runs multi-round buyer ↔ seller negotiation
- All exposed as A2A network services with auto-generated Agent Cards



{ik} INTERVIEW KICKSTART

negotiation_agents/: The Heart of Module 2

```
negotiation_agents/
  buyer_agent/agent.py      → LlmAgent + MCPToolset (pricing only)
  seller_agent/agent.py     → LlmAgent + MCPToolset (pricing + inventory)
  negotiation/agent.py      → LoopAgent(SequentialAgent(buyer, seller))
```

Run the agents:

```
# Chat with them interactively
adk web m2_adk_multiagents/negotiation_agents/

# Expose as A2A network services
adk web --a2a m2_adk_multiagents/negotiation_agents/
```

Before we build this, we'll learn each piece through 9 interactive demos.

Each demo introduces ONE ADK concept - function tools, MCP tools, sessions, sequential, parallel, loop, agent-as-tool, callbacks, event stream.

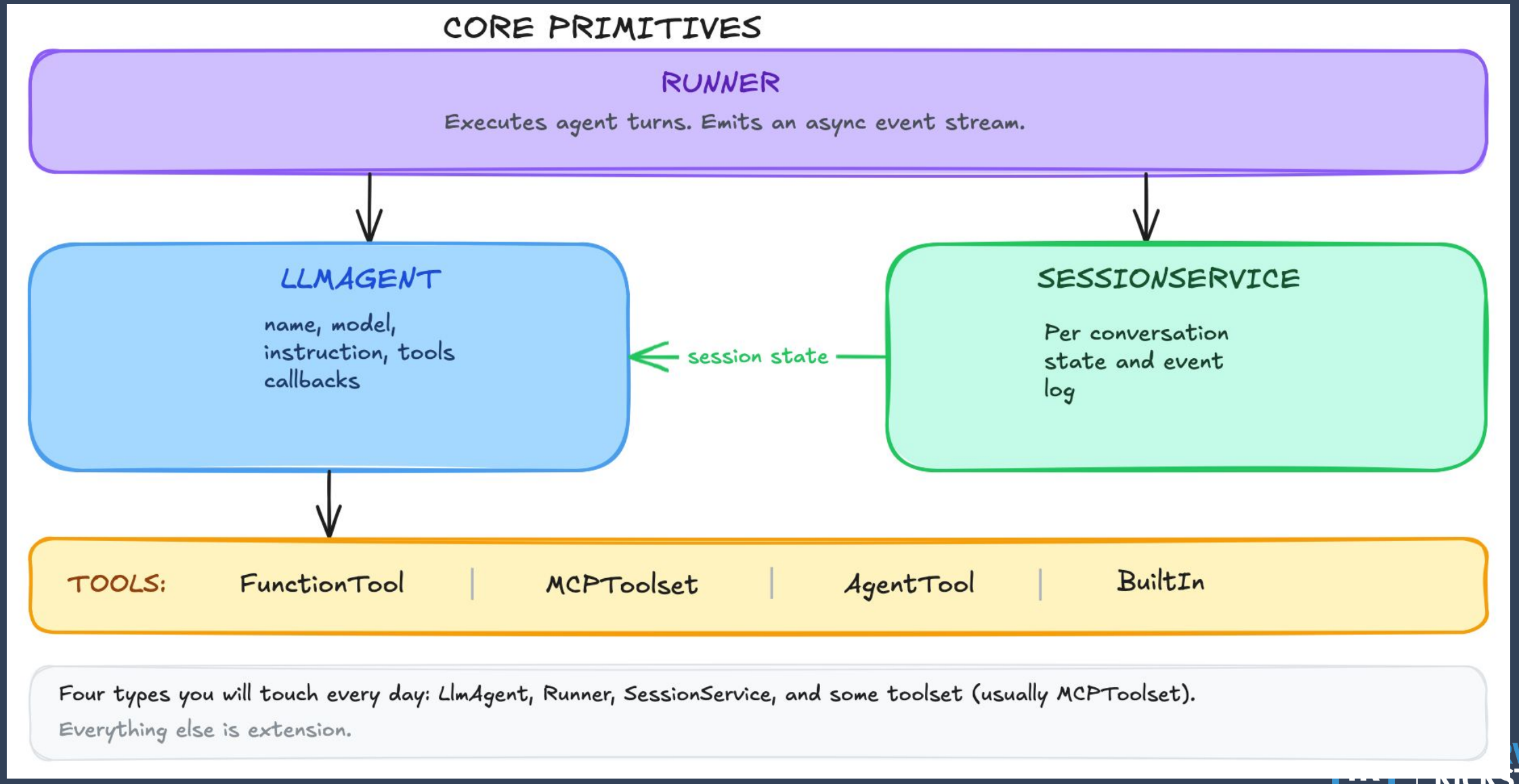
What is Google ADK (Agent Development Kit)

Google's Agent Development Kit (ADK) is an open-source framework that helps developers build, test, evaluate and deploy autonomous AI agents and multi-agent systems. Unlike basic prompt-based approaches, ADK is designed like real software development, with modules, tools, testing and monitoring. It works with any AI model but is deeply integrated with Google's ecosystem including Gemini and Vertex AI. ADK provides easy-to-use building blocks for agents, tools, memory, orchestration patterns, evaluation and deployment.

- **Agent:** the core execution unit i.e autonomous, goal-driven and able to call tools, consult memory and interact with users.
- **Tools:** modular capabilities the agent calls to act in the world (APIs, DBs, other agents, long-running ops). ADK ships with many built-in tools and adapters.
- **Runners / Executors:** components that run agents (local dev server, cloud infra, managed Vertex runtimes).
- **Memory & Artifacts:** mechanisms to store agent state, knowledge and artifacts across sessions (supports retrieval tools).
- **Workflows / Orchestration primitives:** sequential, parallel, loop constructs and patterns for coordinating multiple agents into complex applications.
- **Telemetry & evaluation:** logging, metrics and evaluation harnesses for reproducible testing and monitoring.

{ik} | INTERVIEW KICKSTART

Core Primitives



Demo 01: Basic LlmAgent (Concept)

The Simplest ADK Agent

```
root_agent = LlmAgent(
    name="basic_agent",
    model="openai/gpt-4o",
    instruction="Use get_quick_estimate when asked about properties.",
    tools=[get_quick_estimate],    # plain Python function
)
```

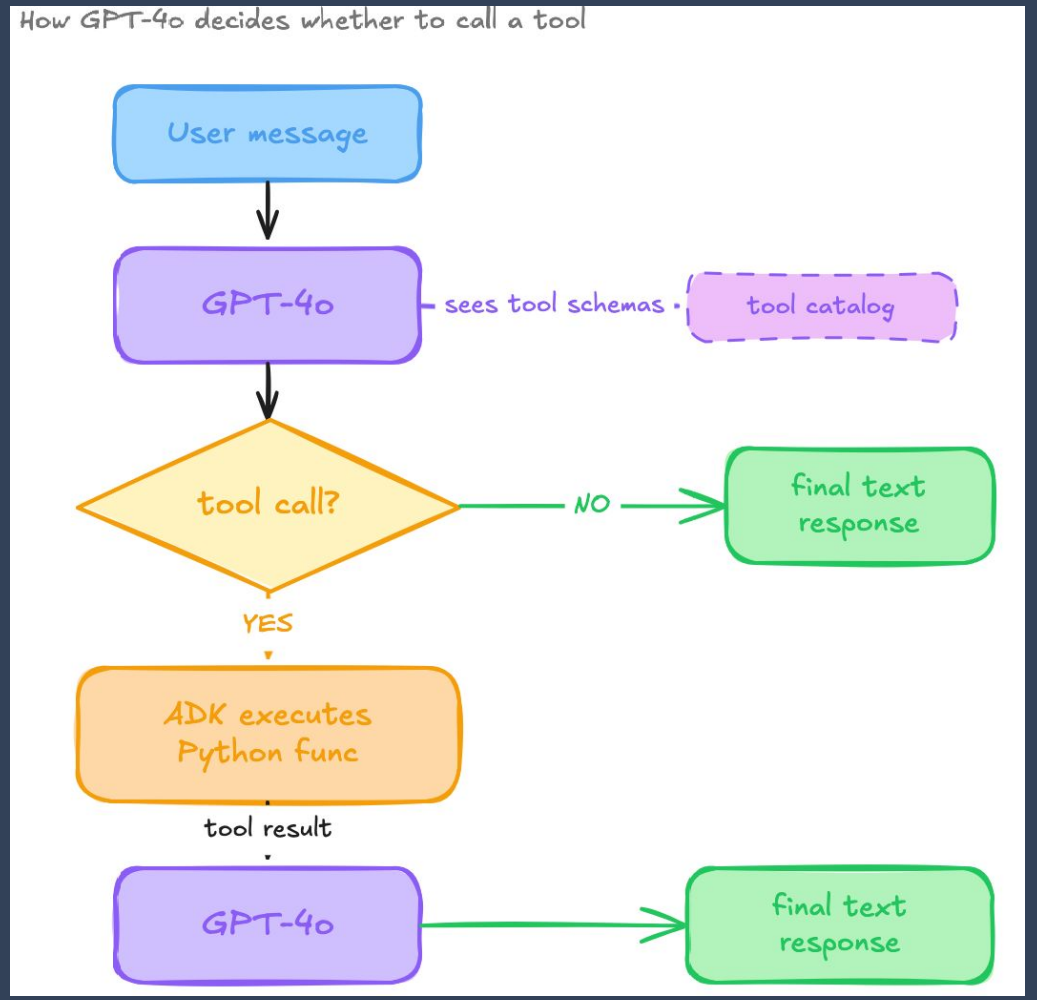
KEY CONCEPTS

- LlmAgent = model + instruction + tools
- root_agent = the variable adk web discovers
- Function tools: ADK reads type hints + docstring → JSON schema
- The LLM decides WHETHER to call tools - not scripted

Try these:

Question	Expected
"What's 742 Evergreen Terrace worth?"	Calls tool → $462K, high confidence
"What about 999 Unknown Ave?"	Calls tool → $450K, LOW confidence
"What's the best neighborhood in Austin?"	NO tool call - LLM knowledge

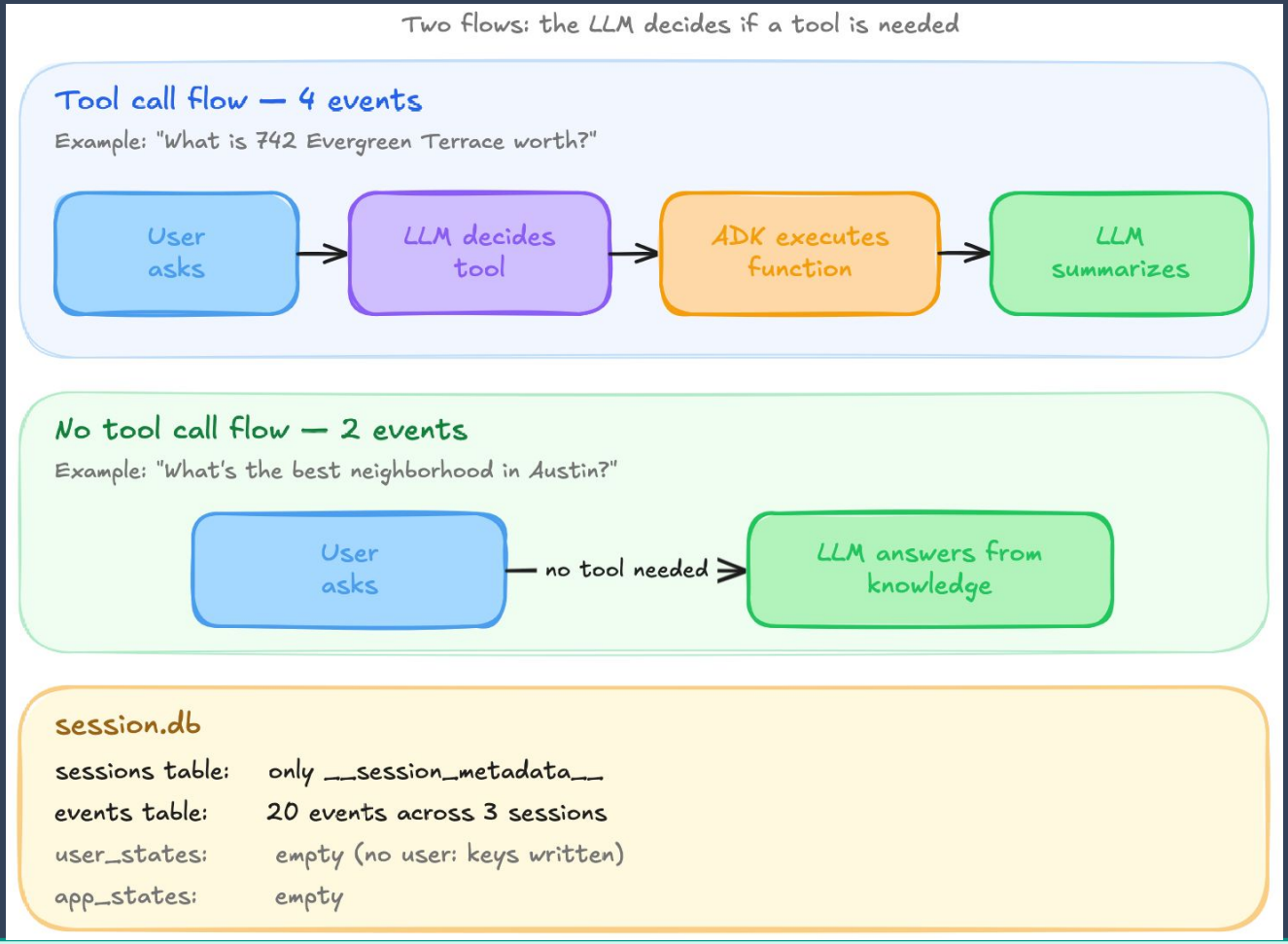
The LLM decides when tools are relevant. Question 3 proves it.



{ik} INTERVIEW KICKSTART

Demo 01: Results

What Happened



KEY TAKEAWAY

Compare to M1's github_agent_client.py - there you hand-wrote the tool loop (call LLM → check for tool_calls → execute → feed back).
Here, tools=[fn] and ADK does the rest.

ADK replaces ~60 lines of manual tool loop with one declaration.

Demo 02: MCP Tools in ADK (Concept)

Auto-Discover Tools from MCP Servers

```
root_agent = LlmAgent(
    name="mcp_tools_agent",
    model="openai/gpt-4o",
    tools=[
        MCPToolset(
            connection_params=StdioConnectionParams(
                server_params=StdioServerParameters(
                    command=sys.executable,
                    args=["m2_mcp/pricing_server.py"],
                ))],
)
```

D01 vs D02

- d01: tools=[get_quick_estimate] - you wrote the function
- d02: tools=[MCPToolset(...)] - ADK spawns server, discovers tools via MCP

Try these:

Question	Expected
"What's 742 Evergreen Terrace worth?"	Calls get_market_price → rich MCP data
"What are the taxes on 742 Evergreen?"	NO tool call - tax tool is commented out

Question 2 is the key moment. The LLM knows it has no tax tool - proves auto-discovery is real.

Demo 02: Results

Auto-Discovery Proven

THE TAX QUESTION PROVES IT

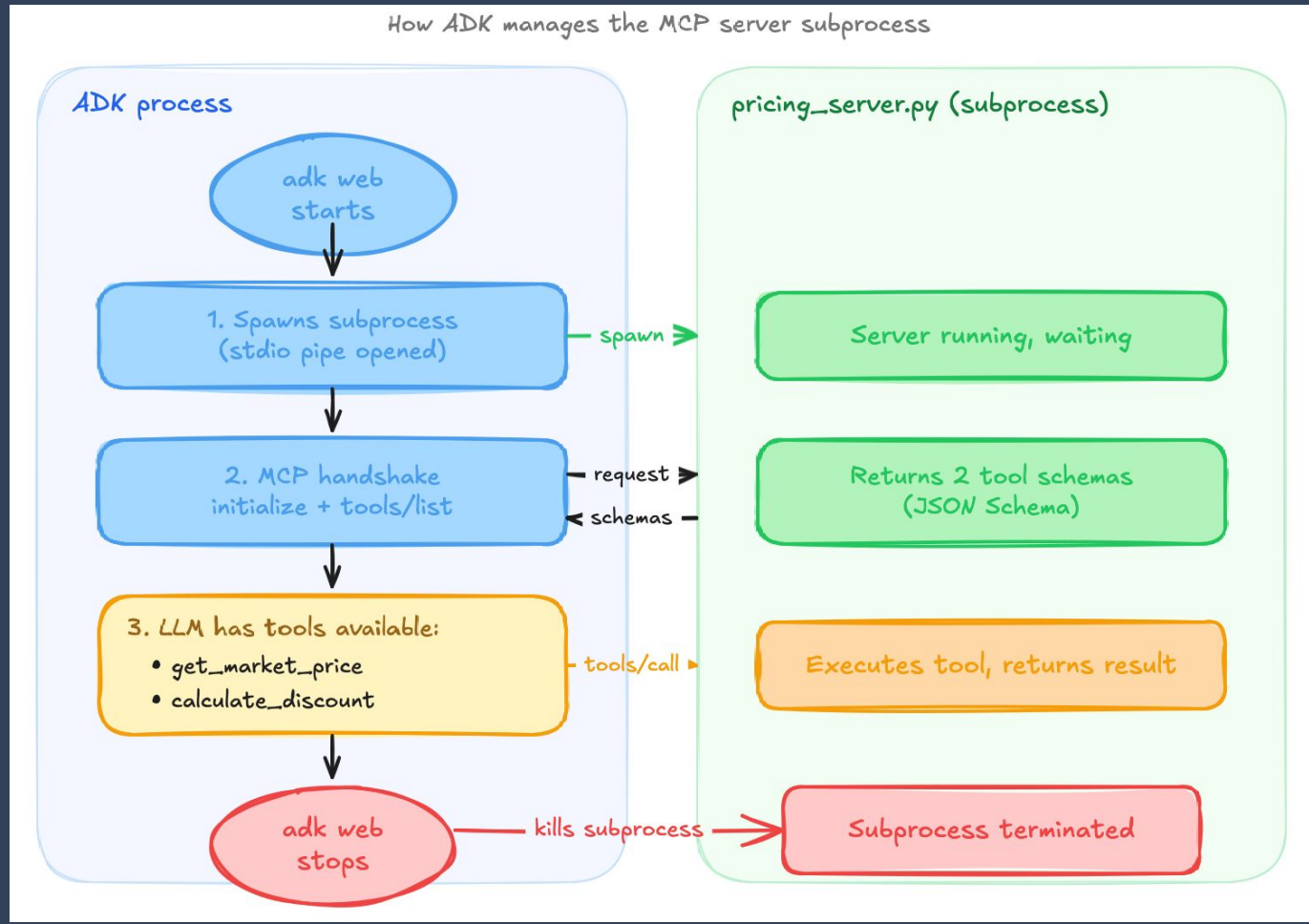
- Instruction does NOT name any tools
- LLM checked its available tools → no tax tool found → declined honestly
- After M2 Exercise 01 (uncomment the tool), the same question works

KEY TAKEAWAY

Add a @mcp.tool() to the server → restart adk web → LLM can call it. Zero agent code changes.

This is what the buyer/seller agents do. Same MCPToolset pattern - just pointed at different servers.

MCPTOOLSET LIFECYCLE



Demo 03: Sessions & State (Concept)

Tools That Remember

```python
def record_offer(price: int, tool_context: ToolContext) -> dict:
    history = tool_context.state.get("offer_history", [])
    history.append(price)
    tool_context.state["offer_history"] = history          # session-scoped
    tool_context.state["user:total_offers"] = len(history) # user-scoped
```

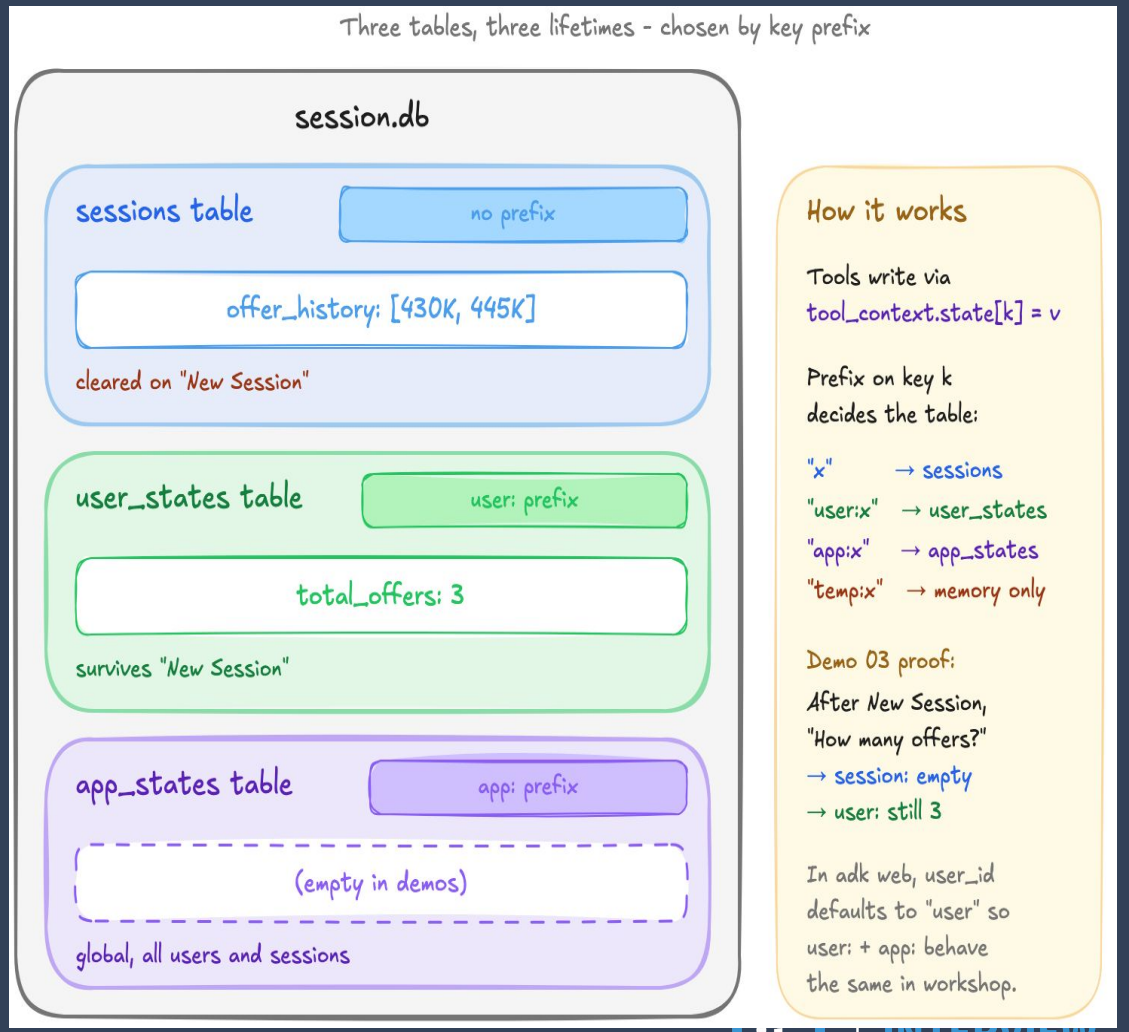
STATE SCOPING

Prefix	Stored in	Lifetime
(none)	sessions table	Cleared on "New Session"
user:	user_states table	Survives across sessions
app:	app_states table	Global, all users
temp:	Memory only	One turn

Try these (in order, same session):

Question	Expected
"Offer $430,000"	Records, total=1
"Offer $445,000"	total=2, best=$445K
"Offer $440,000"	Regression warning!
New Session → "How many offers?"	session=0, total=3

The last question is the proof. Session state resets (offers=[]), but user:total_offers persists (=3). Watch the State tab live.



{ik} INTERVIEW KICKSTART

Demo 03: Results

State Scoping Proven

AFTER "NEW SESSION" → "HOW MANY OFFERS?"

> *"3 offers across all sessions, none in current session"*

Scope	Key	Value	What happened
Session	offer_history	[]	Reset on New Session
User	user:total_offers	3	Persisted!

session.db user_states TABLE - FIRST TIME WITH DATA

```
{"total_offers": 3}
```

Key takeaway: ToolContext.state is the bridge between tools and session memory. State scoping controls what survives.

{ik} | INTERVIEW KICKSTART

Demo 04: SequentialAgent (Concept)

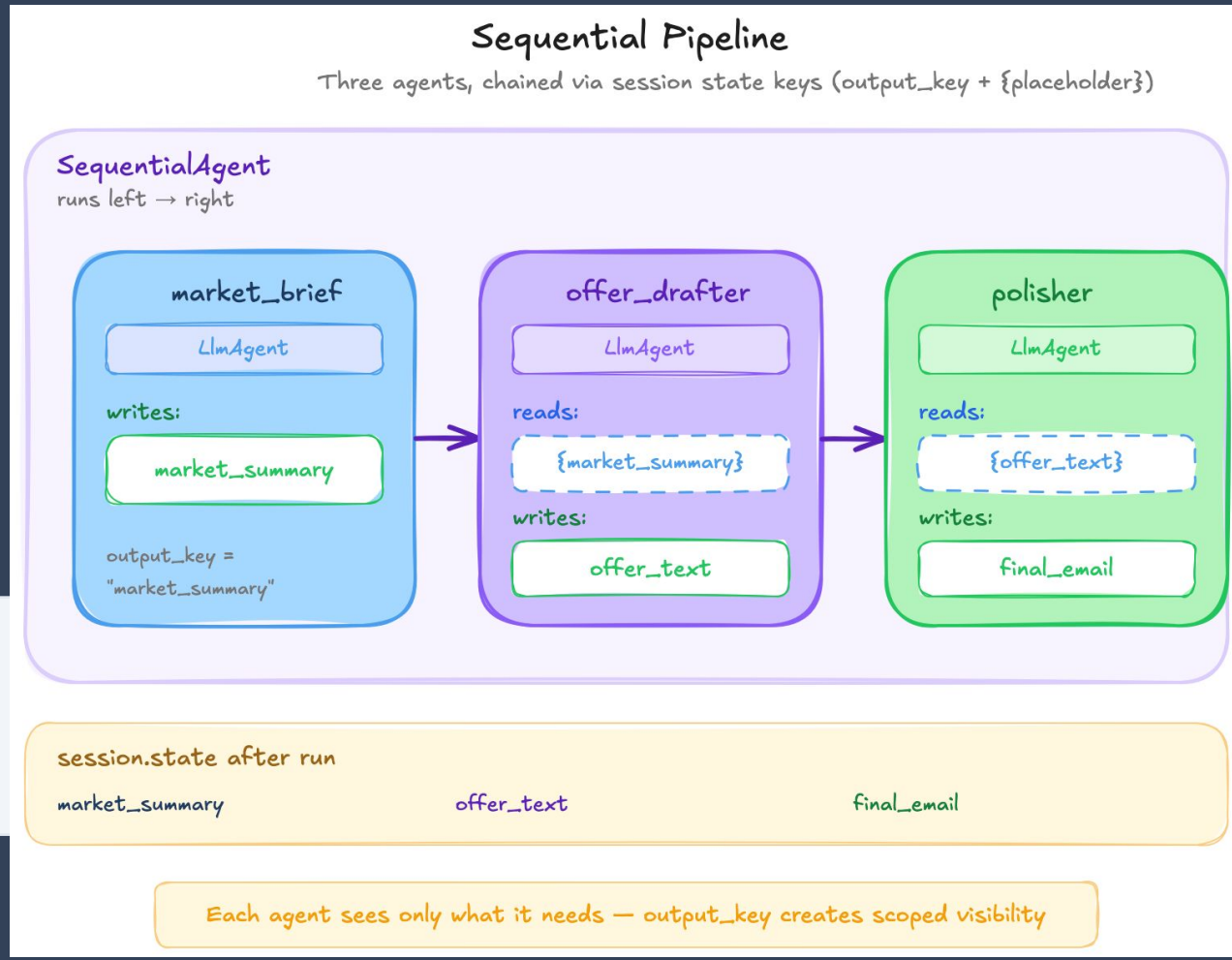
Pipeline: A → B → C

```
market_brief = LlmAgent(output_key="market_summary", ...)
offer_drafter = LlmAgent(output_key="offer_text",
    instruction="Read {market_summary} and draft an offer...")
polisher = LlmAgent(output_key="final_email",
    instruction="Polish {offer_text} into a professional email...")

root_agent = SequentialAgent(
    sub_agents=[market_brief, offer_drafter, polisher],
)
```

DATA FLOW VIA STATE

market_brief writes "market_summary"
→ offer_drafter reads {market_summary}, writes "offer_text"
→ polisher reads {offer_text}, writes "final_email"



{ik} INTERVIEW KICKSTART

Demo 04: Results

Three Agents, One Pipeline

EVENTS (4 TOTAL)

```
Event 1 | user                → "Research this property"
Event 2 | market_brief        → "Austin 78701 median price..."
Event 3 | offer_drafter       → "We extend an offer of $460,000..."
Event 4 | message_polisher    → "Subject: Initial Offer..."
```

STATE TAB AFTER RUN

- market_summary: "Austin 78701..."
- offer_text: "$460K offer..."
- final_email: Full professional email

KEY TAKEAWAY

Each agent saw ONLY what it needed. offer_drafter read {market_summary} - it didn't see the polisher's output. State keys create scoped visibility.

This is half of the negotiation orchestrator. negotiation/agent.py uses SequentialAgent(buyer, seller).

Demo 05: ParallelAgent (Concept)

Fan-Out: A, B, C Run Concurrently

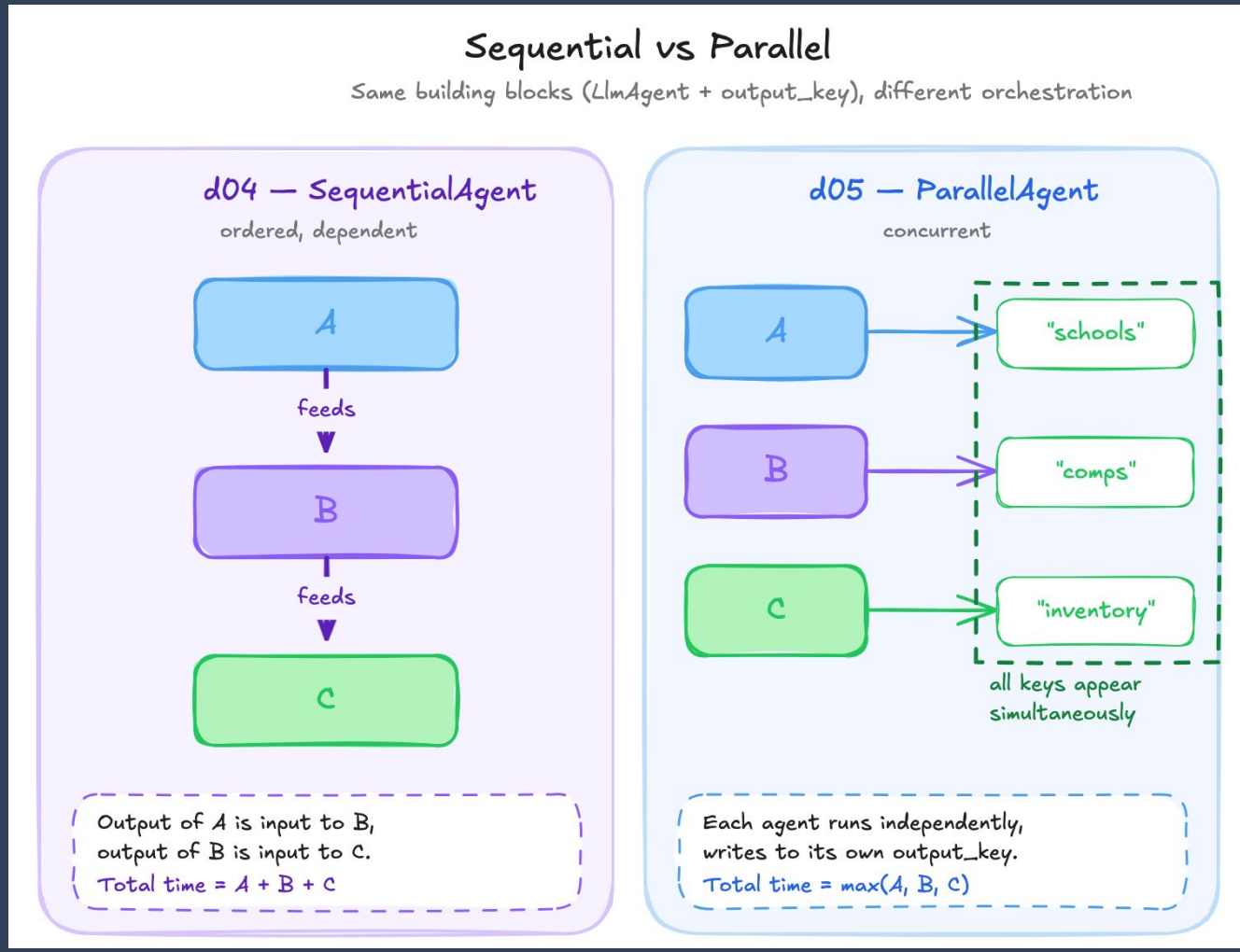
```
schools = LlmAgent(output_key="schools", ...)
comps = LlmAgent(output_key="comps", ...)
inventory = LlmAgent(output_key="inventory", ...)

root_agent = ParallelAgent(
    sub_agents=[schools, comps, inventory],  # all at once
)
```

D04 vs D05

- d04 Sequential: A feeds B feeds C (pipeline)
- d05 Parallel: A, B, C run simultaneously (fan-out)

Rule: Each agent MUST write to a different output_key. Two agents writing the same key → last one wins.



Demo 05: Results

Independent Signals Gathered

STATE TAB AFTER RUN

```
schools:   "Austin ISD has strong academic programs..."
comps:     "Recent sales $700K–$1M range..."
inventory: "High inventory pressure, limited listings..."
```

Same 4-event structure as d04 - but concurrent execution. State key order in the db differs from declaration order (proves concurrency - finish order varies).

KEY TAKEAWAY

ParallelAgent = fan-out for independent tasks. SequentialAgent = pipeline for dependent tasks. Both use output_key.

No {placeholder} reading between agents - they're independent. That's why ParallelAgent works here.

Demo 06: LoopAgent (Concept)

Iterate Until Done

```
def stop_when_in_range(callback_context: CallbackContext):
    price = int(callback_context.state.get("proposal", "0"))
    if 450_000 <= price <= 470_000:
        callback_context.actions.escalate = True  # ← breaks the loop

root_agent = LoopAgent(
    sub_agents=[haggler],
    max_iterations=5,  # bounded - NEVER runs more than 5
)
```

TWO STOP CONDITIONS

- callback_context.actions.escalate = True - early exit (price in range)
- max_iterations=5 - hard cap (same guarantee as M1's FSM)

M1 ↔ ADK CONNECTION

This is the ADK equivalent of M1's FSM termination guarantee. max_iterations = max_turns. escalate = terminal state. Same principle, different abstraction.

{ik} INTERVIEW KICKSTART

Demo 06: Results

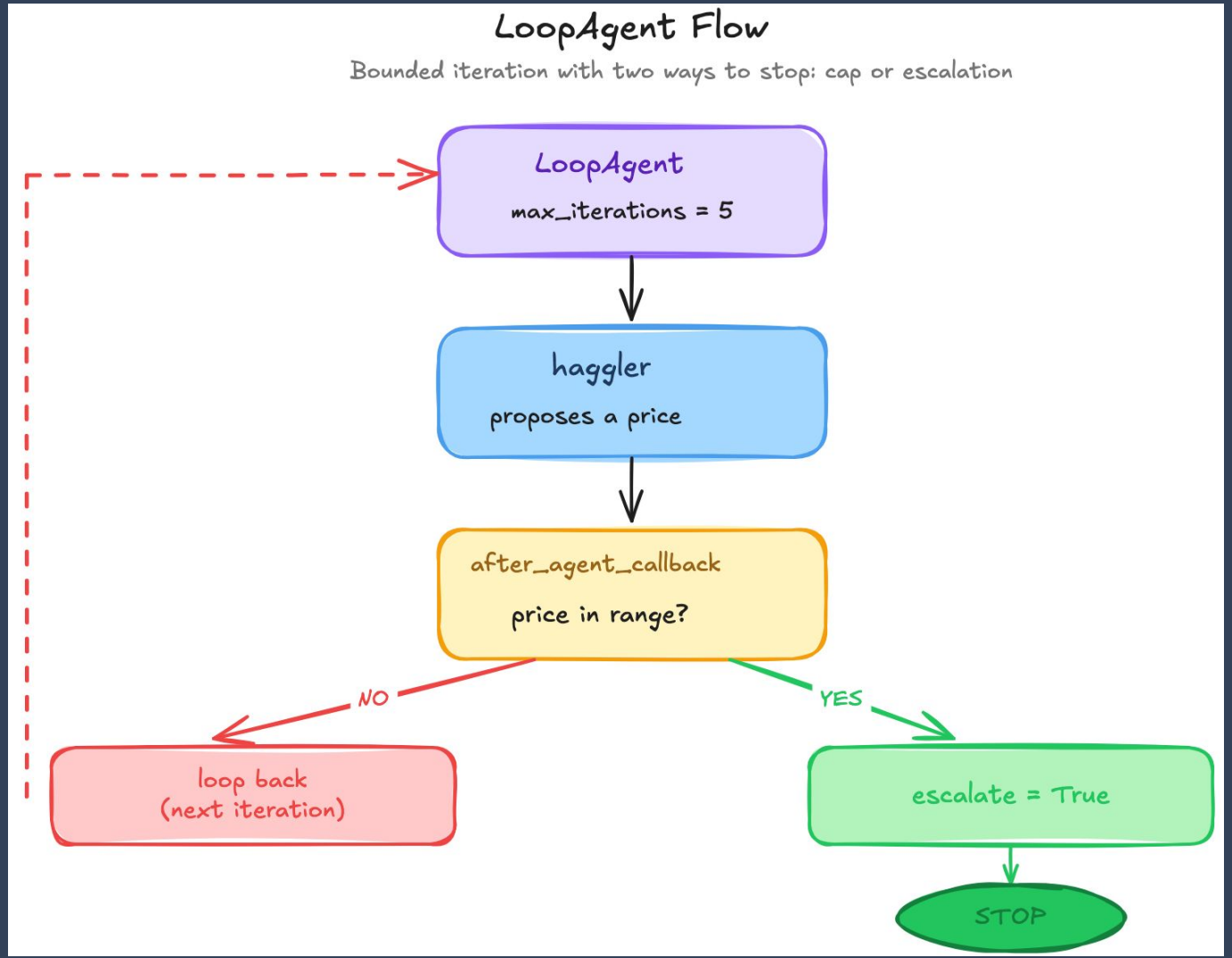
Bounded Termination

ADK LoopAgent
max_iterations
escalate = True
Loop stops
Guarantees termination

output_key in a loop: Each iteration OVERWRITES the same key. Only the last value survives.

LoopAgent = bounded iteration + explicit escalation. No while True, no emergency exits.

negotiation/agent.py uses LoopAgent(SequentialAgent(buyer, seller), max_iterations=5). Each iteration is one round.



{ik} INTERVIEW KICKSTART

Demo 07: Agent-as-Tool (Concept)

Wrap an Agent as a Callable Tool

```
valuator = LlmAgent(
    name="valuator",
    description="Estimates fair market value of Austin properties.",
)

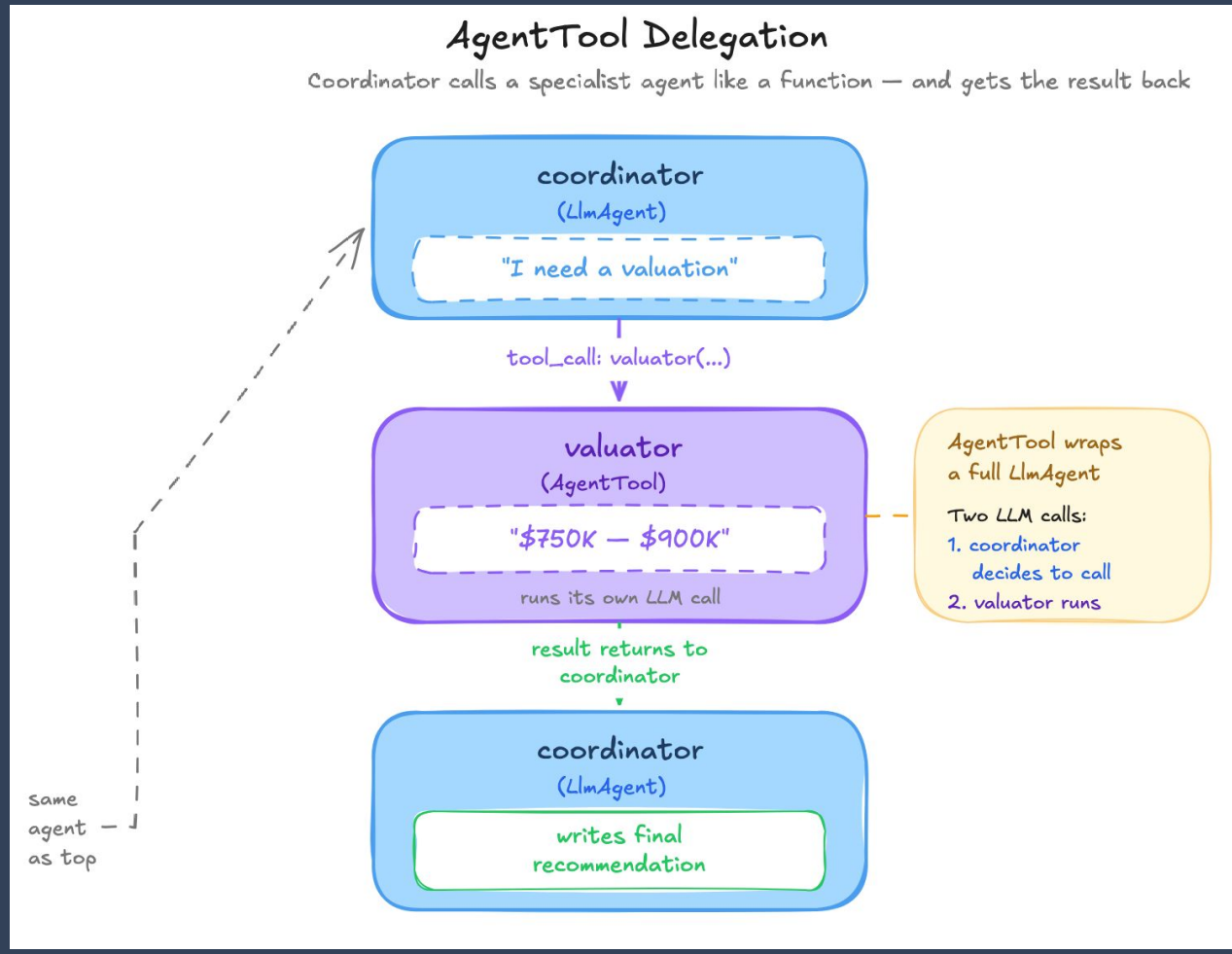
root_agent = LlmAgent(
    name="coordinator",
    tools=[AgentTool(agent=valuator)],  # ← agent wrapped as tool
)
```

THREE DELEGATION PATTERNS

Pattern	How it works	Control
AgentTool	Call agent as function → result returns	Caller keeps control
sub_agents	Transfer to agent → it takes over	Full delegation
SequentialAgent	Fixed order, no runtime choice	No decision

Try:

Question	Expected
"What should I offer on 742 Evergreen?"	Calls valuator → gets estimate → writes recommendation
"What's the weather?"	No valuator call - not relevant



The valuator is a full LlmAgent with its own model call. The coordinator sees it as a function. Two LLM calls per delegation = more expensive but better separation of concerns.

{ik} INTERVIEW KICKSTART

Demo 07: Results

Delegation in Action

EVENT FLOW

```
Event 1 | user        → "What should I offer?"
Event 2 | coordinator → tool_call: valuator({"request": "742 Evergreen..."})
Event 3 | coordinator → tool_result: "$750K–$900K, location is biggest factor"
Event 4 | coordinator → "I recommend offering within $750K–$900K range..."
```

NOTICE

Valuator estimated $750K–$900K (LLM knowledge). d02's MCP-backed get_market_price returned $462K. **This is why MCP matters - without grounded data, the LLM hallucinated a price.**

KEY TAKEAWAY

AgentTool = explicit delegation. The coordinator decides WHEN to call the specialist.

description matters: the coordinator reads valuator.description to decide when to call it - like a function docstring.

Demo 08: Callbacks (Concept)

Policy Without Prompts

Three callback hooks - enforce rules the LLM CAN'T bypass:

Callback	When it fires	What it does
before_model_callback	Before every LLM call	Redact PII (SSN regex)
before_tool_callback	Before every tool call	Block disallowed tools
after_tool_callback	After every tool returns	Log results for audit

RETURN VALUES

- None = allow (continue normally)
- dict = block/replace (short-circuit)

Try (watch terminal, not UI):

Question	Terminal output
"My SSN is 123-45-6789. Estimate 742 Evergreen."	[before_model] redacted PII
"Call get_internal_admin('debug')"	[before_tool] BLOCKED or LLM declines

Callbacks are deterministic - a regex WILL strip the SSN. Instructions are suggestive - the LLM might still leak.

Demo 08: Results

Security Audit Trail

TERMINAL OUTPUT (THE REAL STORY)

```
[before_model] redacted PII from prompt          ← SSN stripped
[before_tool] allow get_quick_estimate(...)      ← on allowlist
[after_tool] get_quick_estimate -> {462000}      ← logged
[before_model] redacted PII from prompt          ← runs AGAIN (history has SSN)
```

before_model fires on EVERY LLM call - not just the first. Each turn has 2+ LLM calls (decide + summarize), and conversation history includes the original SSN message, so redaction runs every time.

KEY TAKEAWAY

Callbacks = deterministic policy. This is what buyer_agent and seller_agent use for tool allowlists (_enforce_buyer_allowlist).
In production, swap print() for structured logging. The audit trail is the compliance layer.

Demo 09: Event Stream (Concept)

See What the Runner Produces

```python
def lookup_comps(address: str, tool_context: ToolContext) -> dict:
    tool_context.state["last_comp_lookup"] = address
    return {"comps": [...], "avg_comp_price": 465_000}

def estimate_offer(comp_avg: int, discount_pct: int, tool_context: ToolContext) -> dict:
    tool_context.state["latest_offer"] = int(comp_avg * (1 - discount_pct / 100))
    tool_context.state["offer_count"] = tool_context.state.get("offer_count", 0) + 1
```

FOCUS ON

- State tab (updates live)
- Left panel event inspector (state deltas)

Try: "What should I offer on 742 Evergreen Terrace?" → check State tab

This demo is about seeing state writes and event structure, not the chat response.

{ik} | INTERVIEW KICKSTART

Demo 09: Results

Parallel Tool Pitfall

WHAT HAPPENED

LLM called BOTH tools in parallel:

```
lookup_comps({"address": "742 Evergreen"})     → returned avg: $465K
estimate_offer({"comp_avg": 300000, ...})      → used $300K (WRONG!)
```

The LLM made up $300K because it decided both arguments BEFORE seeing either result. Nobody told it to parallelize - GPT-4o chose to emit both calls in one response.

The fix: SequentialAgent (d04) when tool B depends on tool A's output, or more explicit instructions.

KEY TAKEAWAY

Parallel tool calls are great for independent tasks (d05). Dangerous when there are data dependencies.

This is a known GPT-4o behavior. The instruction said "call lookup, then estimate with the result" but the model ignored the dependency.

No Question Is Too Small Every Question Deserves an Answer!!

Ask about anything:

- Concepts from today's modules
- How this applies to your work
- Implementation details
- Clarifications on what we covered

Quick Request

- Please do not mind if a question gets skipped accidentally
- Repost politely in chat if needed
- We will try our best to cover as many questions as possible

Remember: If you're wondering about it, others probably are too.



Time for a Break



A 5-minute break is starting now. Please return when the timer ends to continue the presentation

{ik} | INTERVIEW KICKSTART

Live Zoom Poll



We are launching a poll on Zoom. Please take 2 minutes to fill out the form.

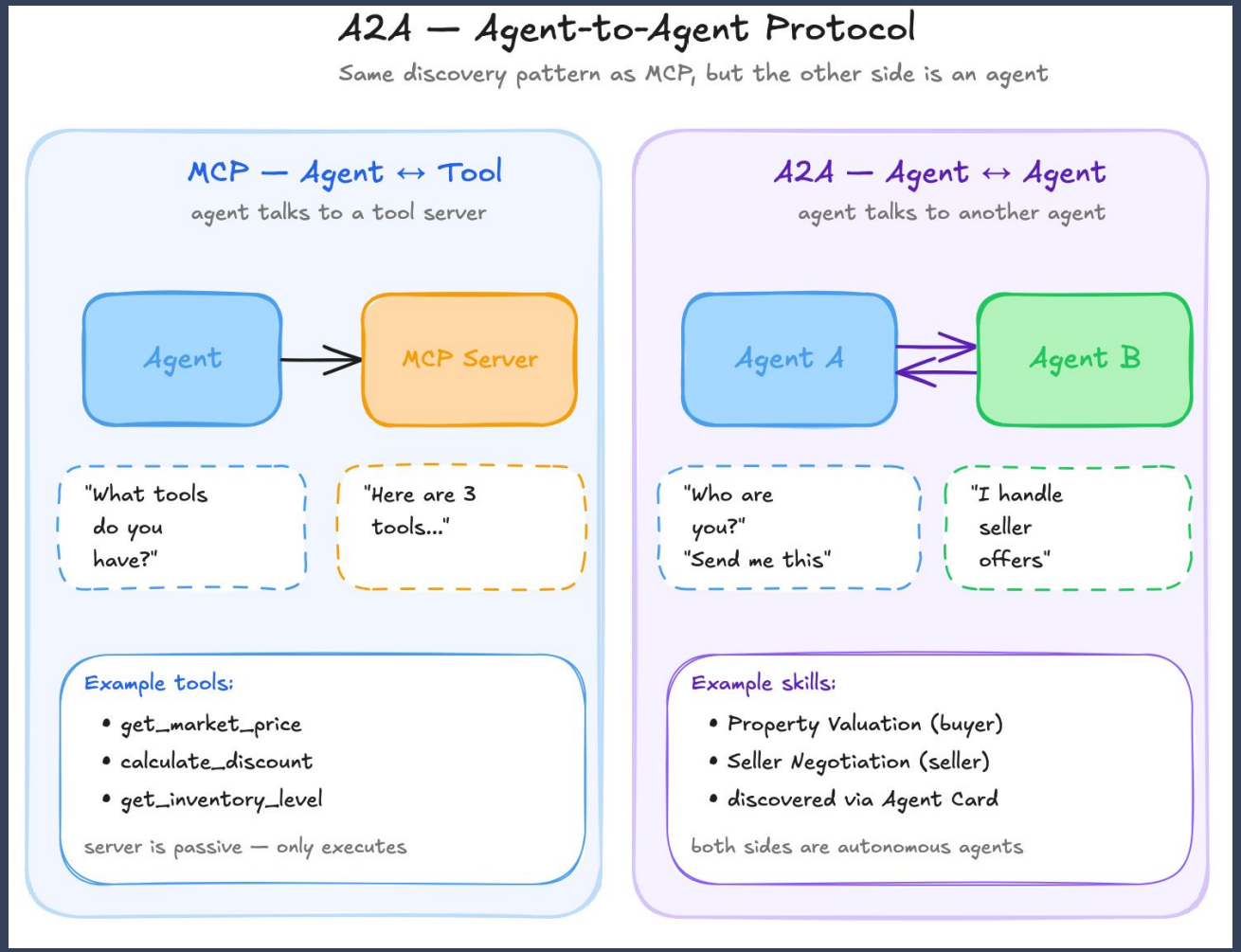
{ik} | INTERVIEW KICKSTART

What is A2A ?

Agent-to-Agent Protocol

The problem: So far, all our agents run in the SAME process. The buyer and seller are Python objects in the same adk web instance. What if they're on different machines, different teams, different languages?

A2A solves this. It's an open protocol for agents to discover and communicate over HTTP.



THREE OPERATIONS (SAME PATTERN AS MCP)

- Discovery: GET /.well-known/agent-card.json - "Who are you and what can you do?"
- Schema: Agent Card - name, description, skills, capabilities
- Invocation: POST / (JSON-RPC message/send) - "Handle this task"

{ik} INTERVIEW KICKSTART

Why A2A Matters

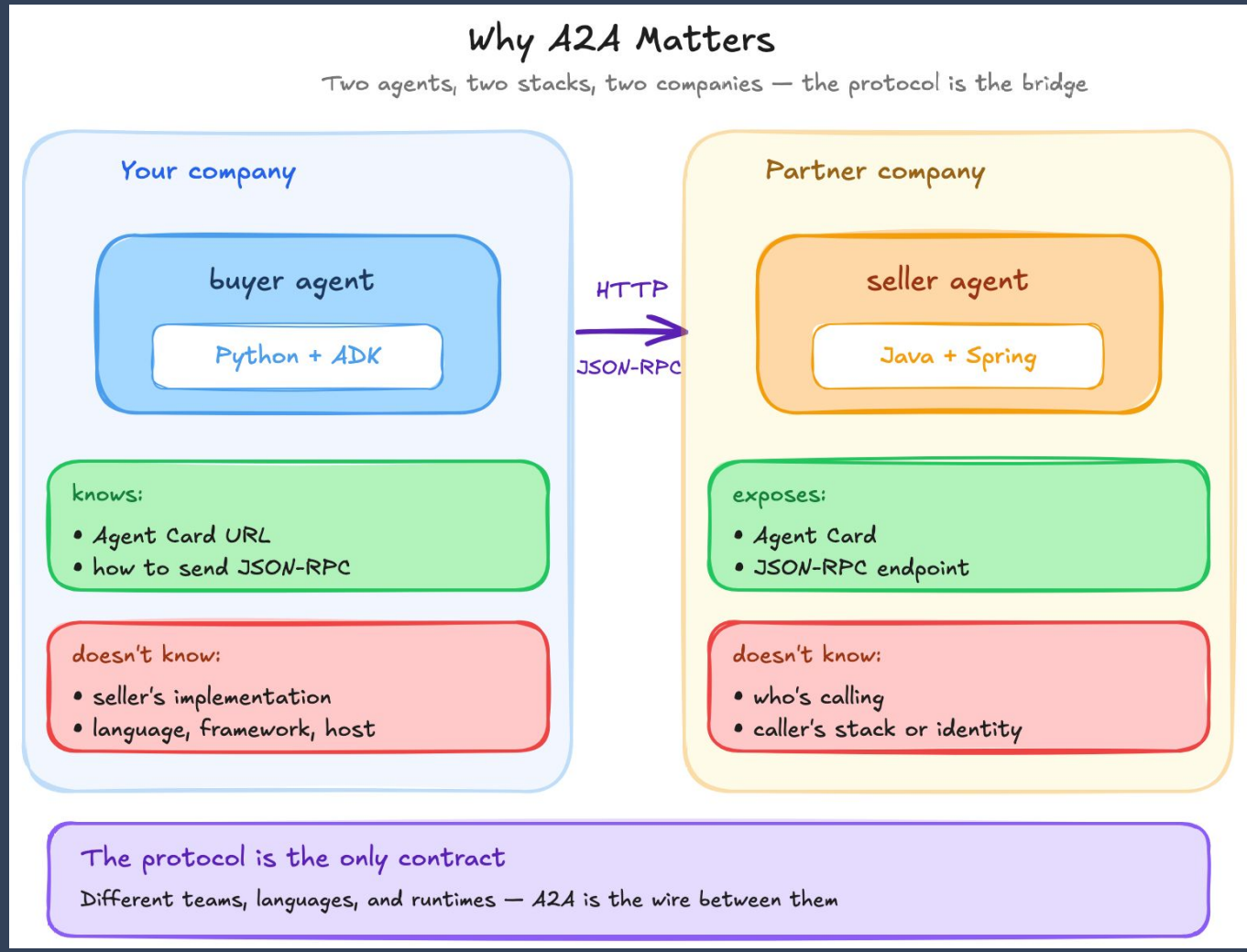
From In-Process to Network Services

WITHOUT A2A (D01–D09)

- Buyer and seller are Python objects in the same process
- They communicate via shared session state (output_key, {placeholder})
- Tightly coupled - you must import both to run either

WITH A2A

- Buyer runs on Machine A, seller runs on Machine B
- They communicate via HTTP JSON-RPC
- Neither knows how the other is built - only the Agent Card
- Could be different languages, different teams, different companies



{ik} INTERVIEW KICKSTART

Agent Card

Agent Card = Agent's Business Card

Every A2A agent publishes a JSON document at /.well-known/agent-card.json:

```
{
  "name": "seller_agent",
  "description": "Real estate seller agent for 742 Evergreen Terrace",
  "url": "http://localhost:8000/a2a/seller_agent",
  "capabilities": {"streaming": true},
  "skills": [{
    "id": "negotiation_response",
    "name": "Real Estate Seller Negotiation",
    "description": "Responds to buyer offers with counter-offers",
    "tags": ["real_estate", "seller", "negotiation"]
  }]
}
```

WHAT A CLIENT LEARNS FROM THE CARD

- What this agent does (description + skills)
- Where to send messages (url)
- What features are supported (streaming, push notifications)
- No documentation needed - self-describing, like MCP tool schemas

{ik} | INTERVIEW KICKSTART

Demo 10: A2A Wire Format (Concept)

Raw JSON-RPC + Task Lifecycle

THREE STEPS

- GET /a2a/seller_agent/.well-known/agent-card.json - discover the agent
- POST /a2a/seller_agent (valid offer $440K) - task: completed, counter at $477K
- POST /a2a/seller_agent (broken envelope) - task: still completed (LLM handled it)

```
{"jsonrpc": "2.0", "method": "message/send",
 "params": {"message": {"parts": [{"kind": "text", "text": "...offer JSON..."}]}}}
```

RESPONSE CONTAINS

- status: "completed" - the agent processed the request
- contextId - reuse for round 2 (demo 11)
- artifacts - the counter-offer text as a durable output
- history - full message exchange

Key insight: completed means the PROTOCOL worked - not that the negotiation succeeded.

Demo 10: Results

What the Wire Looks Like

VALID OFFER RESPONSE

```
status: "completed"
contextId: "4f0fff3e-..."     <- reuse for round 2
artifacts: [{text: "Counter-offer at $477,000..."}]
history: [Message(user), Message(agent)]
```

BROKEN ENVELOPE RESPONSE

```
status: "completed" (NOT failed!)
artifacts: [{text: "Could you please resend your offer?"}]
```

Why still completed? The A2A protocol layer worked correctly - valid JSON-RPC, valid Message. The content was garbage, but the LLM coped. Tasks only fail on protocol errors or server crashes.

Field	What it contains
status	Task state: completed/failed
contextId	Thread ID for multi-turn (demo 11)
history	All Messages exchanged
artifacts	Durable outputs (the counter-offer)

Demo 11: Context Threading (Concept)

Multi-Turn Negotiations via contextId

Three rounds, same contextId, the seller remembers everything:

Round	Offer	Seller's Response
1	$432K	Below floor ($445K). Counter at $477K.
2	$440K	STILL below $445K. Consider $477K.
3	$446K	Above $445K. We ACCEPT! Congratulations!

```
Round 1: POST -> server assigns contextId
Round 2: POST + contextId -> seller remembers round 1
Round 3: POST + contextId -> seller sees full history -> ACCEPTS
```

Without contextId, the seller would counter at $477K every time - no memory of prior offers.

{ik} INTERVIEW KICKSTART

Demo 11: Results

Why Memory Matters

SAME contextId ACROSS ALL 3 ROUNDS

```
4498d50f-a61e-4a17-8c84-0ea13d49a283
```

The seller accepted in round 3 because it remembered:

- Round 1: MCP tool returned floor price = $445K
- Round 2: buyer came up from $432K to $440K (still below)
- Round 3: $446K > $445K → ACCEPT immediately

KEY TAKEAWAY

contextId = A2A's session_id. It ties independent HTTP requests into one coherent conversation. The same concept as ADK's session state (d03), but across network boundaries.

{ik} | INTERVIEW KICKSTART

Demo 12: Parts & Artifacts (Concept)

Multi-Part Messages + Durable Outputs

Sent a message with TWO Parts - same offer in two formats:

```
parts[0] = TextPart("Final-and-best at $445k")     <- human-readable
parts[1] = DataPart({hint: "machine copy", offer: {...}})  <- structured JSON
```

Part type	Purpose	Who reads it
TextPart	Human-readable text	The LLM
DataPart	Structured JSON	Code / downstream systems
FilePart	Binary (PDF, image)	Not used in this demo
Artifact	Durable output on Task	Anyone who fetches the Task later

Parts = conversational (in Messages). **Artifacts = deliverables** (on Tasks).

{ik} | INTERVIEW KICKSTART

Demo 12: Results

Acceptance at $445K

RESPONSE HISTORY SHOWED THE FULL AGENT PROCESSING

- User message: 2 parts (TextPart + DataPart)
- Agent MCP tool calls: 3 DataParts (get_market_price, get_minimum_acceptable_price, calculate_discount)
- Agent tool results: 3 DataParts (structured responses)
- Agent final text: "We accept your offer of $445,000!"

ARTIFACT - SAME ACCEPTANCE TEXT AS DURABLE OUTPUT

```
{"artifactId": "f7ae8903-...",
 "parts": [{"kind": "text", "text": "We accept..."}]}
```

$445K = exact floor → immediate acceptance. MCP tool confirms min = $445K. Offer matches. Seller accepts.

Tool calls appear as DataParts in history - you can programmatically inspect what tools were called.

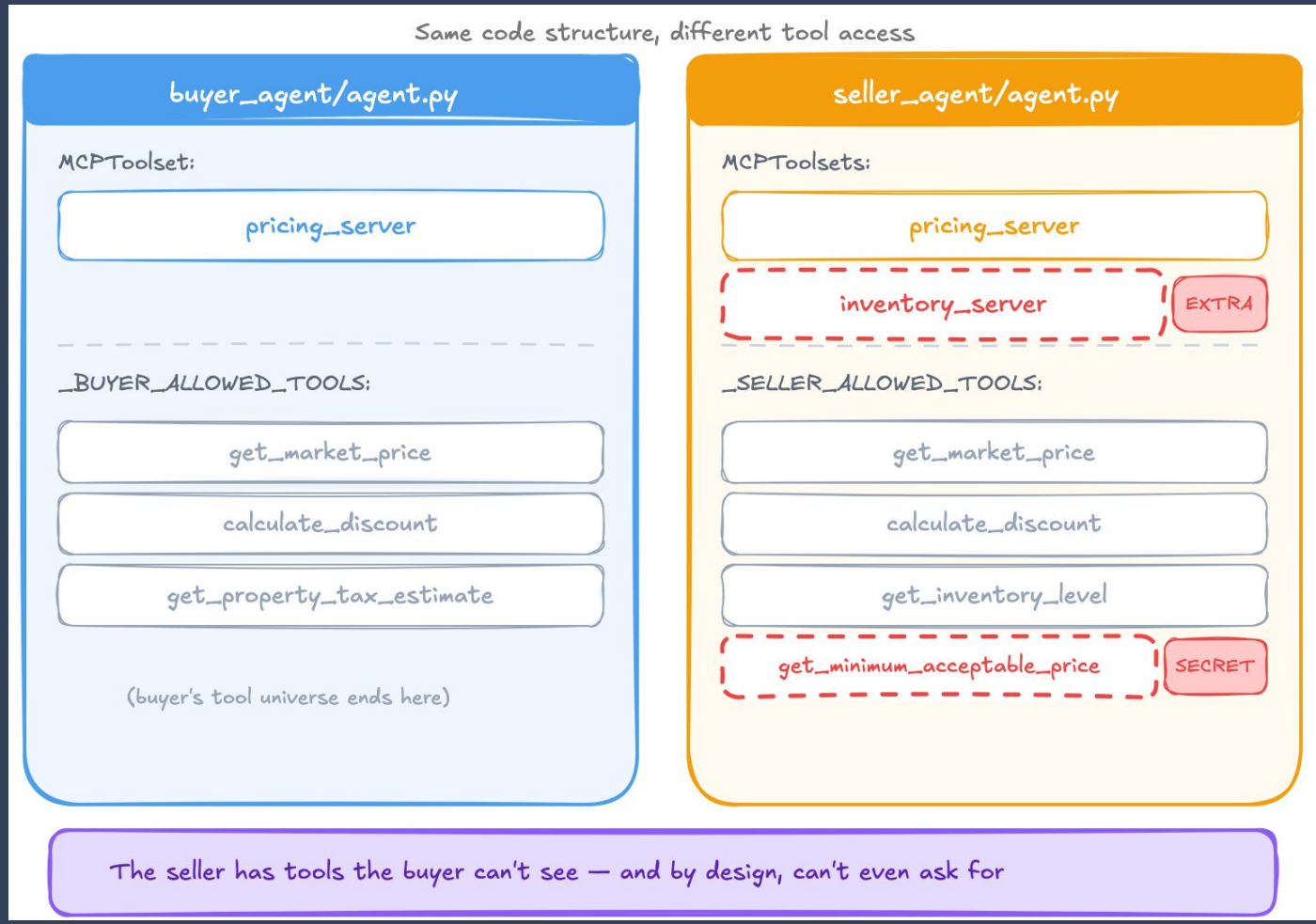
Buyer vs Seller: Information Asymmetry

Information Asymmetry - The Design Point

What each side knows:		
Data	**Buyer**	**Seller**
Market price ($462K)	Yes	Yes
Discount analysis	Yes	Yes
Inventory levels	No	Yes
Floor price ($445K)	No	Yes

HOW IT'S ENFORCED

- before_tool_callback = allowlist (d08 pattern)
- Even if the LLM tries to call get_minimum_acceptable_price, the callback blocks it
- The LLM can't bypass a callback - it's deterministic, not suggestive



Buyer & Seller: Results

Buyer & Seller - What We Observed

BUYER AGENT (8 EVENTS, 2 QUERIES)

```
Query 1: "What's 742 Evergreen Terrace worth?"
  -> get_market_price -> $462K estimated value, $485K list price
  -> "The estimated market value is $462,000..."

Query 2: "I want to make an offer. What do you recommend?"
  -> calculate_discount(base=485K, balanced, 18 DOM, good condition)
  -> "I recommend an initial offer of ~$425,000 (12% below asking)"
```

SELLER AGENT (6 EVENTS, 2 QUERIES)

```
Query 1: "Buyer offers $440,000 with 30-day close"
  -> 3 tools called IN PARALLEL:
    get_market_price + get_inventory_level + get_minimum_acceptable_price
  -> "Offer below our minimum of $445,000. COUNTER at $477,000."

Query 2: "Buyer increases to $446,000"
  -> NO tool calls (remembered floor from query 1)
  -> "$446,000 is above $445,000. ACCEPT."
```

Seller's parallel calls are independent (no data dependency) - unlike d09 where parallel was dangerous.

Negotiation Orchestrator: Composition

The Orchestrator - Every Demo in One File

```python
# negotiation/agent.py - the complete composition
def submit_decision(action: str, price: int, tool_context: ToolContext) -> dict:
    tool_context.state["seller_decision"] = {"action": action.upper(), "price": price}
    return {"recorded": action.upper(), "price": price}

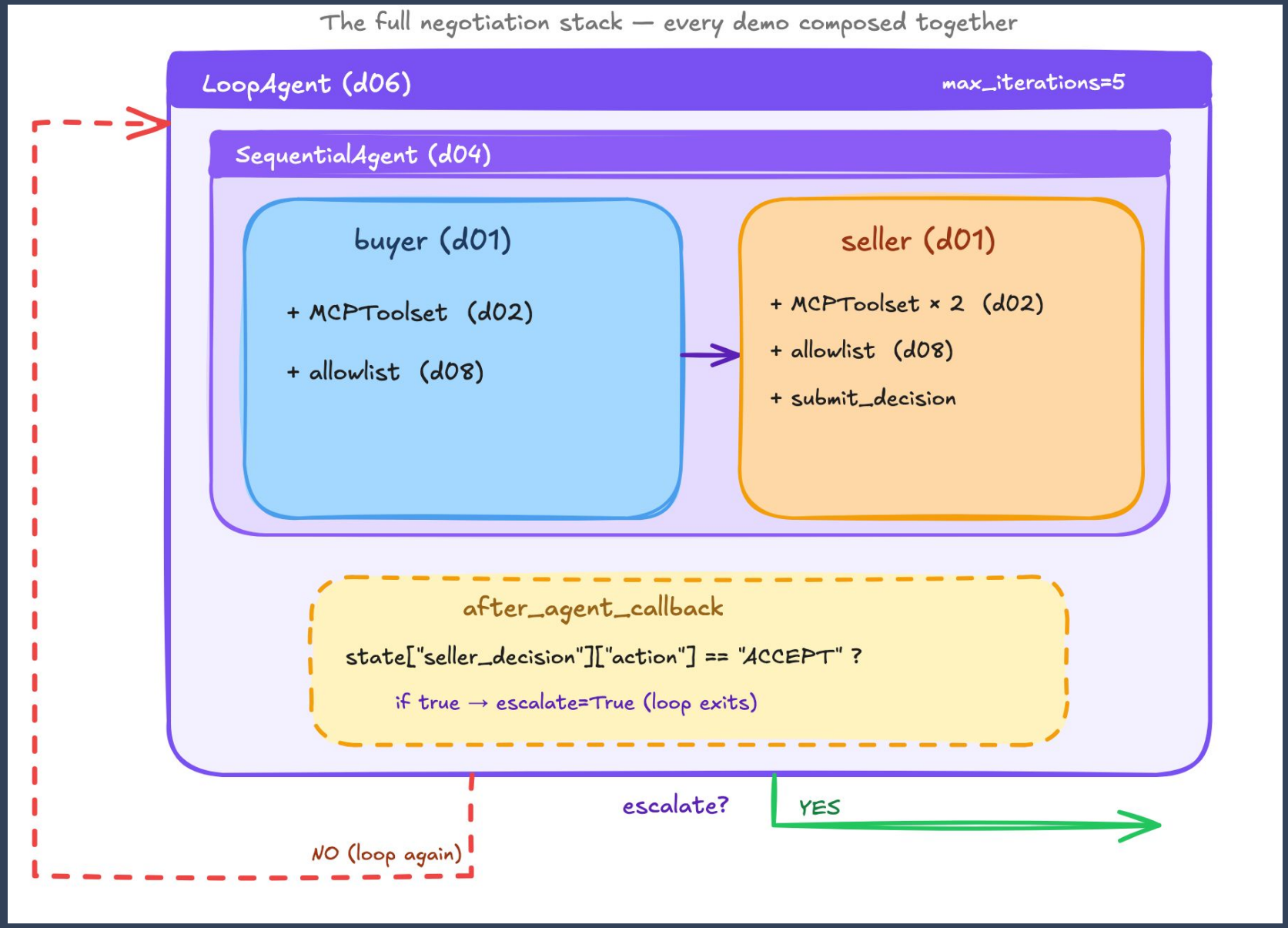
def _check_agreement(callback_context):
    decision = callback_context.state.get("seller_decision")
    if isinstance(decision, dict) and decision.get("action") == "ACCEPT":
        callback_context.actions.escalate = True

buyer = LlmAgent(output_key="buyer_offer", tools=[MCPToolset(pricing)], ...)
seller = LlmAgent(output_key="seller_response",
    tools=[MCPToolset(pricing), MCPToolset(inventory), submit_decision],
    after_agent_callback=_check_agreement, ...)
root_agent = LoopAgent(sub_agents=[SequentialAgent([buyer, seller])],
    max_iterations=5)
```

Concept	Where
LlmAgent (d01) + MCPToolset (d02)	buyer, seller
output_key + {placeholder} (d04)	buyer writes -> seller reads
SequentialAgent + LoopAgent (d04, d06)	round + max iterations
Callbacks (d08) + submit_decision tool	Allowlists + structured signal

{ik} INTERVIEW KICKSTART

Diagram: How It All Connects



{ik} INTERVIEW KICKSTART

Negotiation: Live Run Results

Negotiation - What Actually Happened

2 negotiation rounds → agreement at $445,000:

Round	Buyer (MCP tools)	Seller (MCP tools)	Decision
1	$425,800 (get_market_price + calculate_discount)	get_min + get_inventory → COUNTER 477K	COUNTER $477K
2	$445,000 (no tools - jumped to target)	submit_decision(ACCEPT, 445000)	ACCEPT $445K

```
Event 1          | user   → "Start negotiation"
Event 3          | buyer  → tool_call: get_market_price + calculate_discount
Event 4          | buyer  → tool_results (market data)
Event 5          | buyer  → "Offer: $425,800..."          → state["buyer_offer"]
Event 6          | seller → tool_call: get_minimum_acceptable_price + get_inventory_level
Event 7          | seller → tool_results (floor=$445K, hot market)
Event 8          | seller → "COUNTER $477,000" + tool_call: submit_decision(COUNTER, 477000)
Event 9          | seller → tool_result: {recorded: COUNTER}  → callback: no escalation
Event 10         | seller → confirms counter
Event 11         | buyer  → "Offer: $445,000..."          → overwrites buyer_offer
Event 12         | seller → "Accepted" + tool_call: submit_decision(ACCEPT, 445000)
Event 13         | seller → tool_result: {recorded: ACCEPT}  → callback: ESCALATE!
Event 14         | seller → confirms acceptance
Event 15-20  | buyer/seller → post-acceptance chatter (current iteration completes)
```

OBSERVATIONS

- MCP tools grounded every decision - buyer used $462K market data, seller used $445K floor
- submit_decision is the structured signal - callback reads dict, not free text
- Buyer jumped from $425K to $445K in round 2 with confidence from MCP data
- Post-acceptance chatter still occurs - escalation stops NEXT iteration, not current

STATE TAB AFTER COMPLETION


```
{"seller_decision": {"action": "ACCEPT", "price": 445000}}
```


{ik} INTERVIEW KICKSTART

A2A Orchestrated Negotiation

Full Negotiation Over A2A

```
# Terminal 1:
adk web --a2a m2_adk_multiagents/negotiation_agents/

# Terminal 2:
python m2_adk_multiagents/a2a_13_orchestrated_negotiation.py
```

The script discovers BOTH agents via Agent Cards, then orchestrates multi-round negotiation over HTTP:

```
Step 1: Discover agents
  GET buyer_agent/.well-known/agent-card.json   -> skills: [Property Valuation]
  GET seller_agent/.well-known/agent-card.json  -> skills: [Seller Negotiation]

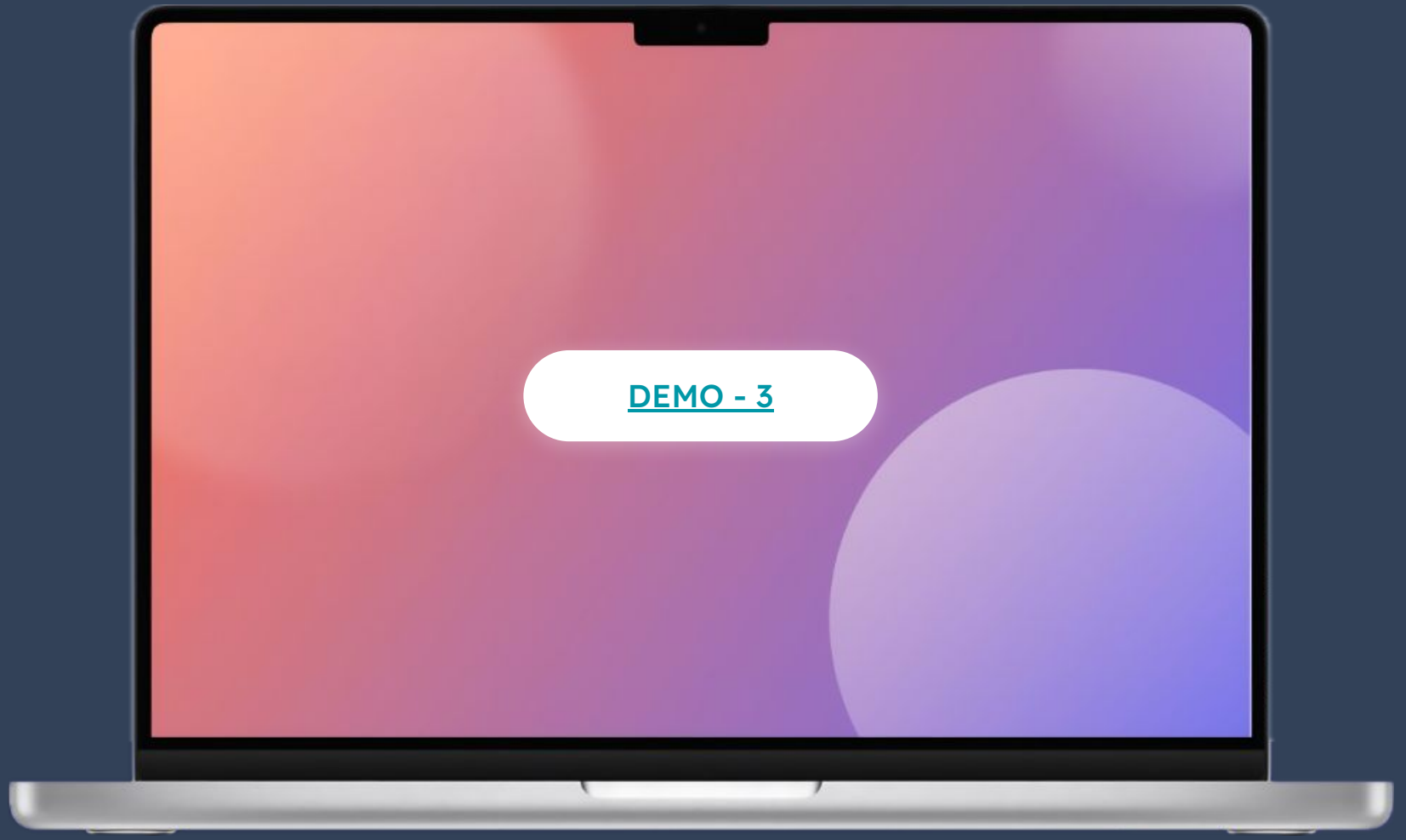
Step 2: Multi-round negotiation
  Round 1: POST buyer_agent  -> "Offer $425K"    (buyer calls MCP tools)
           POST seller_agent -> "COUNTER $477K"  (seller calls MCP tools)
  Round 2: POST buyer_agent  -> "Offer $445K"
           POST seller_agent -> "ACCEPT $445K"   -> DEAL REACHED
```

KEY A2A DETAILS

- Separate contextIds per agent - script acts as matchmaker; agents don't know about each other
- Each agent calls its own MCP tools - buyer: pricing, seller: pricing + inventory
- 4 total A2A messages for a complete negotiation

{ik} INTERVIEW KICKSTART

Module 2 - Implementation Notes: Google ADK + A2A Protocol



{ik} | INTERVIEW KICKSTART

No Question Is Too Small Every Question Deserves an Answer!!

Ask about anything:

- Concepts from today's modules
- How this applies to your work
- Implementation details
- Clarifications on what we covered

Quick Request

- Please do not mind if a question gets skipped accidentally
- Repost politely in chat if needed
- We will try our best to cover as many questions as possible

Remember: If you're wondering about it, others probably are too.





{ik} | INTERVIEW KICKSTART